自动舵控制算法开发文档

> **文档编号**：AR-DEV-2026-003 **版本**：V1.0 **编制日期**：2026-07-30 **密级**：内部公开 **上游文档**：《自动舵定位与控制设计文档》（AR-DOC-2026-002） **配套文档**：《自动舵控制算法 C++ 代码说明文档》（AR-CODE-2026-001） **代码目录**：`control/include/`

目录

1. 概述与开发总纲
2. 开发流程总体规划
3. PD 航向控制器开发
4. 增益调度开发
5. Nomoto 模型辨识开发
6. 控制参数自动优化（AutoTuner）开发
7. 机动序列（ManeuverSequencer）开发
8. 参数存储与标定流程开发
9. 核心集成与主循环开发
10. 测试与验证计划
11. 可视化调参工具开发
12. 实船调试接口开发

- 附录 A：术语与符号

---

1. 概述与开发总纲

1.1 编制目的

本文档面向 **开发工程师**，将《自动舵定位与控制设计文档》中 §3（PD 航向控制、标定、自动优化）、§4（仅航向机动）、§5（联合工作流）的设计要求，拆解为可执行的开发任务：明确每个算法模块的实现细节、边界条件、易错点、开发顺序与测试验收标准。

三份文档的分工关系：

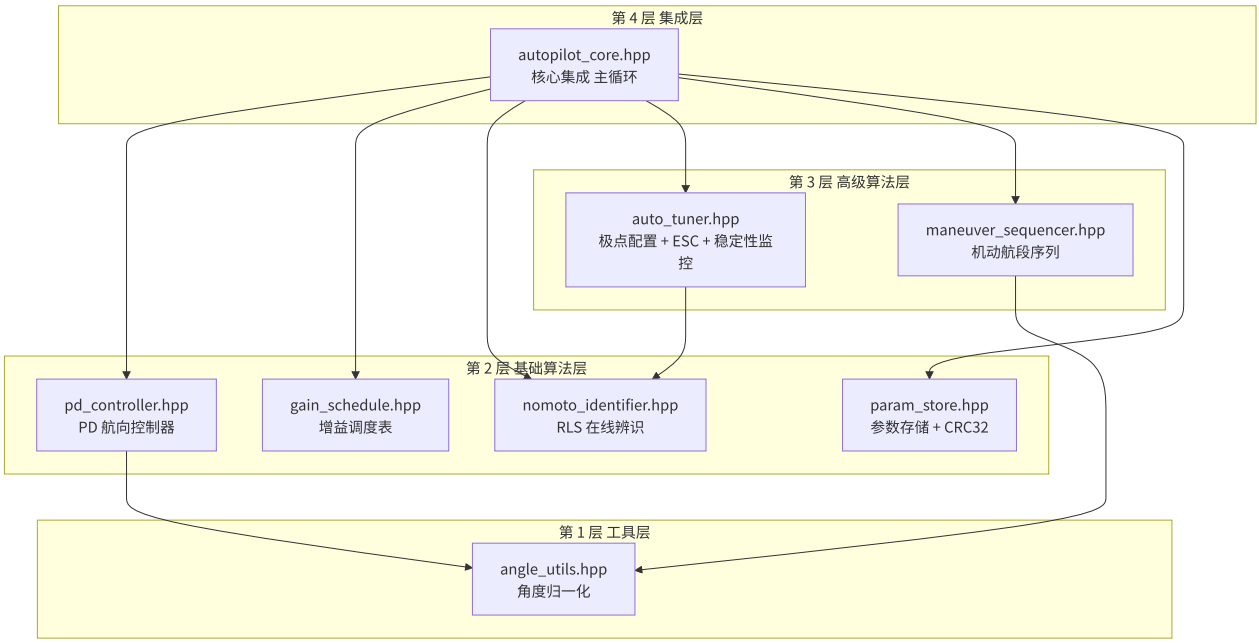
文档	视角	回答的问题
设计文档（AR-DOC-2026-002）	系统设计	做什么、为什么这样做
本开发文档（AR-DEV-2026-003）	**开发实现**	**怎么拆、怎么写、怎么验**
代码说明文档（AR-CODE-2026-001）	API 使用	接口怎么调、参数是什么

1.2 开发范围

覆盖 `control/include/` 下 8 个纯算法模块的开发全过程，以及配套的 **可视化调参工具**（`control/ui/debug_ui.html`，作为开发/试验支撑项目，见 §11）与 **实船调试接口及命令行调试工具**（`control/include/tuner_debug_api.hpp` + `control/tools/`，见 §12）。**不包含**：GNSS 定位解算（设计文档 §2，由定位组独立开发）、硬件 IO / BSP 驱动、实船驾驶台人机界面。

1.3 模块分层与依赖

模块间依赖自底向上分为四层，开发顺序遵循依赖方向（先底层后上层）：



1.4 开发原则

1. **纯算法层**：所有模块不含任何硬件 IO（GPIO/CAN/UART/EEPROM），传感器输入与舵机输出一律通过函数参数传递；EEPROM 读写通过 `std::function` 注入。这是单元测试与 HIL 仿真可行的前提。
2. **header-only + C++17 + 零第三方依赖**：便于直接嵌入 ECU 工程，避免交叉编译链接问题。

3. **确定性执行**：主循环内禁止动态内存分配（`ManeuverSequencer::setLegs` 等配置类接口除外）、禁止异常、禁止阻塞调用，保证 50 Hz（20 ms）周期的最坏执行时间可控。
4. **单位与约定统一**：角度一律用 **度（deg）**，转向率 deg/s，船速 kn，时间 s；航向误差恒定义为 `ref − act` 并归一化到 (−180°, 180°]；舵角右舵为正。
5. **可追溯**：每个函数/常量注释标注对应设计文档章节号（如 `§3.3`），设计变更时可反向检索受影响代码。

1.5 设计输入汇总（关键指标）

开发与验收所依据的设计指标（摘自设计文档 §1.3、§3.8、§3.9.6）：

指标	要求	验证阶段
控制周期	20 ms（50 Hz 解算）	单元测试 / HIL
航向控制精度	⩽ ±1.0°（静水）/ ±2.0°（2 级海况），1σ	海试
阶跃超调	⩽ 10%	闭环仿真 / 海试
5% 调节时间	⩽ 30 s	闭环仿真 / 海试
稳态航向误差	⩽ 1°	闭环仿真 / 海试
操舵频次	⩽ 6 次/分钟	闭环仿真 / 海试
闭环阻尼比	ζ ⩾ 0.7（监控下限 0.6）	仿真 / 在线监控
闭环自然频率	ωn ⩽ 0.5 rad/s	仿真 / 在线监控
舵角/舵速限幅	±35°、10°/s	单元测试
辨识拟合残差	RMS < 5% · r_max	仿真 / 海试

1.6 追溯矩阵（设计 → 开发 → 代码 → 里程碑）

设计文档章节	本文档章节	代码文件	里程碑
§3.2 控制律 / 角度归一化	§3.3~§3.4（DP-1/DP-2）	`pd_controller.hpp` / `angle_utils.hpp`	M1
§3.3 输出限幅	§3.4（DP-4/DP-5）	`pd_controller.hpp`	M1

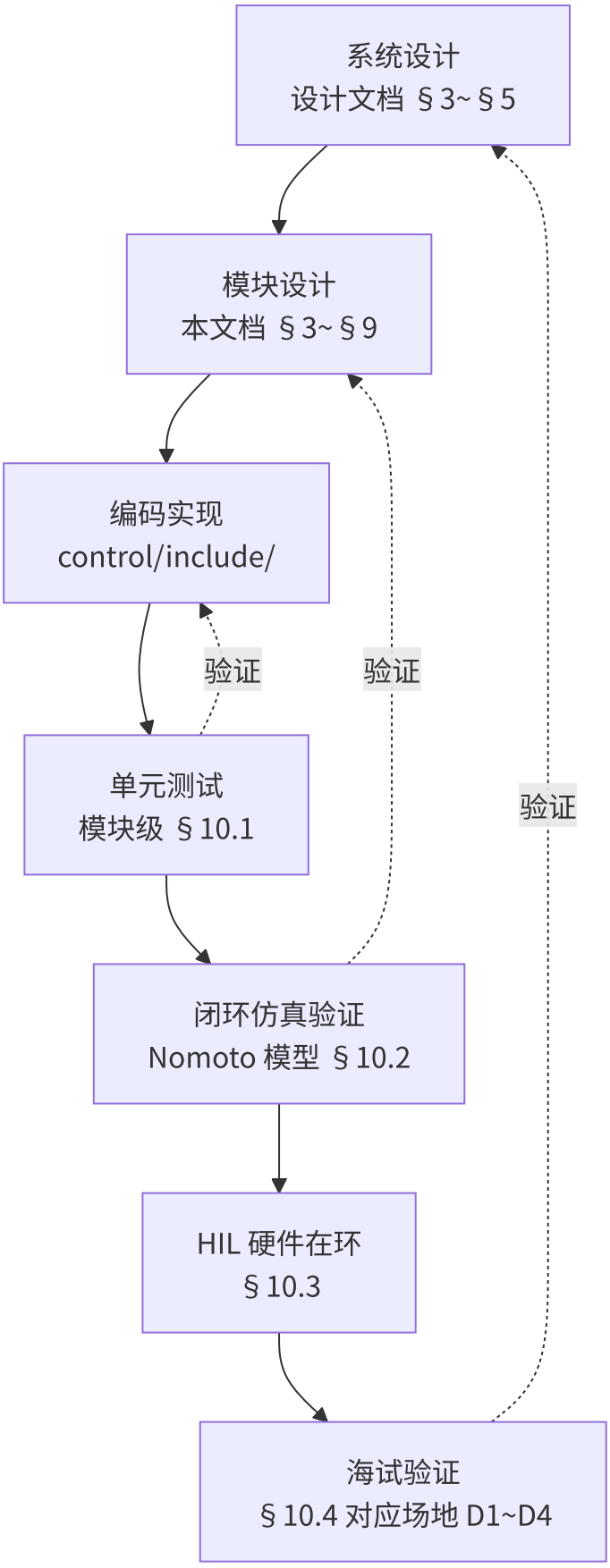
设计文档章节	本文档章节	代码文件	里程碑
§3.7 抗噪与死区	§3.4（DP-2/DP-3/DP-6）	`pd_controller.hpp`	M1
§3.5~§3.6 参数整定 / 增益调度	§4	`gain_schedule.hpp`	M1
§3.4 Nomoto 模型	§5.2	`nomoto_identifier.hpp`	M2
§3.9.4 Z 形试验辨识	§5.3（路线 A）	离线辨识工具	M2
§3.9.3 / §3.9.8 执行器标定与存储	§8	`param_store.hpp`	M2
§3.9.5 PD 增益初算	§6.3	`auto_tuner.hpp`（`polePlacement`）	M3
§3.10.3 RLS 在线辨识	§5.4~§5.5	`nomoto_identifier.hpp`	M2
§3.10.4 极点配置重算	§6.3	`auto_tuner.hpp`	M3
§3.10.5 ESC 性能寻优	§6.4	`auto_tuner.hpp`（`escStep`）	M3
§3.10.6~§3.10.7 触发 / 稳定性监控	§6.4.3、§6.5	`auto_tuner.hpp` / `autopilot_core.hpp`	M3/M5
§4.2 机动调度框架	§7.2~§7.3	`maneuver_sequencer.hpp`	M4
§4.3~§4.8 六种机动	§7.4	`maneuver_sequencer.hpp`	M4
§5 联合工作流	§9	`autopilot_core.hpp`	M5
§3.9.6 性能指标 / §7.11 海试日程	§10.2、§10.4	仿真环境 / 试验大纲	M1~M5

---

2. 开发流程总体规划

2.1 V 模型开发流程

整体采用 V 模型：左侧自顶向下细化设计与编码，右侧自底向上逐级验证，每个开发阶段与一个验证阶段一一对应。

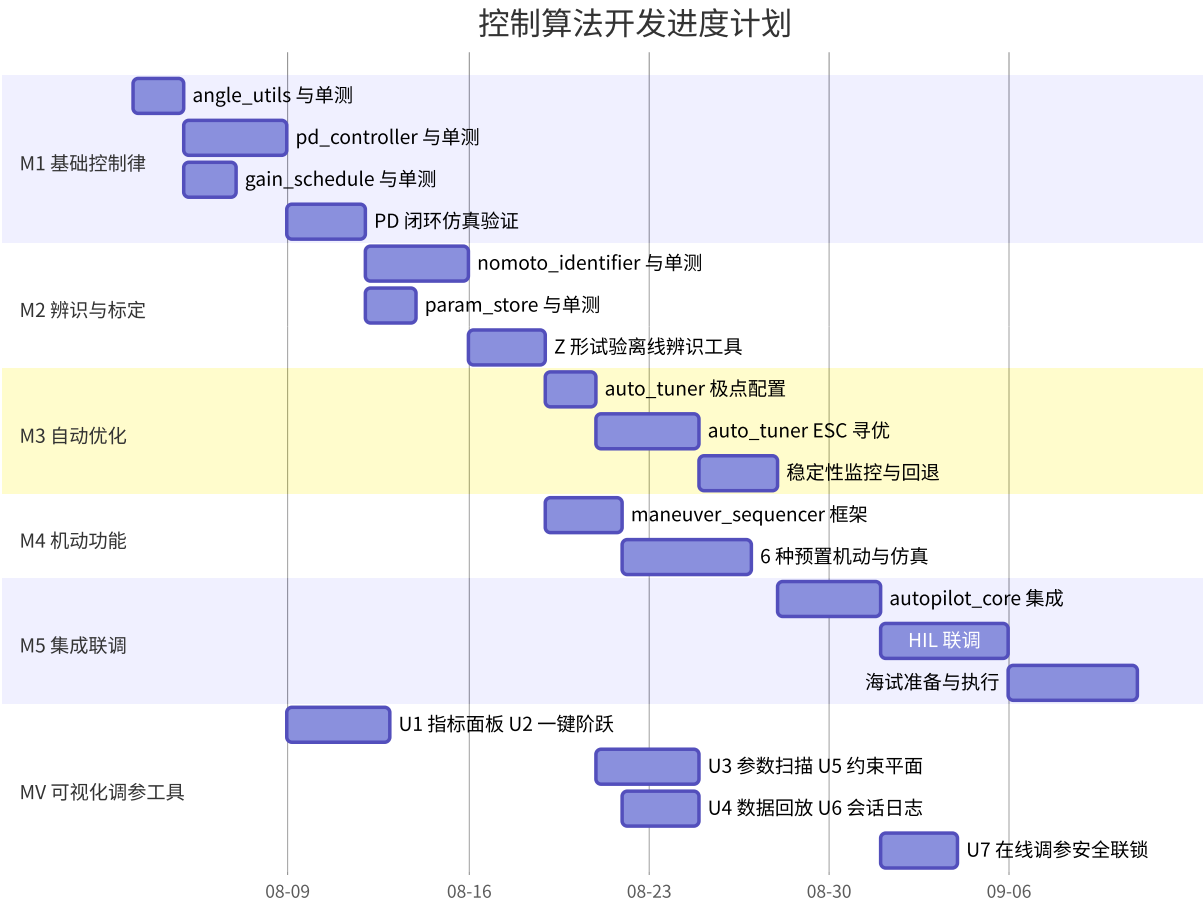

系统设计
设计文档 §3~§5
模块设计
本文档 §3~§9
编码实现
control/include/
验证
验证
单元测试
模块级 §10.1
验证
闭环仿真验证
Nomoto 模型 §10.2
HIL 硬件在环
§10.3
海试验证
§10.4 对应场地 D1~D4

2.2 里程碑划分

按模块依赖关系与风险从低到高，划分 5 个里程碑：

里程碑	内容	交付物	准出标准
M1 基础控制律	`angle_utils` + `pd_controller` + `gain_schedule`	三模块 + 单测	单测全过；PD 阶跃仿真超调 ⩽ 10%
M2 辨识与标定	`nomoto_identifier` + `param_store` + Z 形试验离线工具	两模块 + 单测 + 辨识脚本	已知 K/T 仿真数据收敛误差 < 5%；CRC 读写回环通过
M3 自动优化	`auto_tuner`（极点配置 + ESC + 稳定性监控）	模块 + 单测	极点配置反推增益仿真达标；ESC 在仿真中 J 单调收敛；异常参数触发回退
M4 机动功能	`maneuver_sequencer` + 6 种预置机动	模块 + 单测	6 种机动在仿真中航段推进正确、轨迹形态符合 §4
M5 集成联调	`autopilot_core` + 模式切换 + HIL	集成 + HIL 报告	HIL 全模式切换无异常；50 Hz 周期最坏执行时间 < 2 ms
MV 可视化调参工具（贯穿）	`debug_ui.html` 增强：指标面板、一键试验、参数扫描、回放、会话日志、在线联锁（详见 §11.4 U1~U7）	UI V1.1 + 使用手册更新	U1/U2 随 M1 交付、U3/U5 随 M3、U4/U6 随 M4、U7 随 M5；验收判据见 §11.6

2.3 开发进度计划



MV 各任务与对应算法里程碑并行：U1/U2 在 PD 编码完成后立即启动（M1 的 PD 仿真验证即用它出报告），U7 必须在 HIL 联调前就绪（HIL/海试的在线调参依赖联锁保护）。

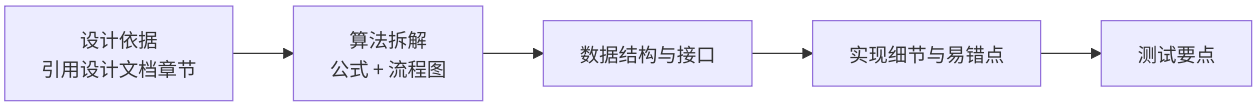
2.4 开发环境与工具链

项	选型	说明
语言标准	C++17	与 ECU 交叉编译器能力对齐
宿主编译器	GCC ⩾ 9 / Clang ⩾ 10	单测与仿真在 PC 上运行
编译选项	`-Wall -Wextra -Werror -O2`；单测加 `-fsanitize=address,undefined`	尽早暴露未定义行为
单测框架	GoogleTest 或 Catch2（header-only，与工程风格一致）	每模块一个测试文件

项	选型	说明
仿真被控对象	Nomoto 一阶模型 + 可选舵机一阶惯性 + 量测噪声	见 §10.2
可视化调参工具	`control/ui/debug_ui.html`（增强计划见 §11.4）	参数编辑、实时曲线、指标评估、在线调参
实船调试接口与工具	`tuner_debug_api.hpp` + `ar_tuner_debug` + tools 监控/验收脚本（§12）	单环节激励-观测、实时监控、端到端验收
版本管理	Git；算法参数变更须在提交信息中注明对应文档章节	可追溯

2.5 模块开发统一模板

第 3~9 章每个模块按统一结构展开，开发时按此顺序推进：



---

3. PD 航向控制器开发

3.1 设计依据

设计文档 §3.2（控制律与角度归一化）、§3.3（输出限幅）、§3.7（抗噪与死区）。实现文件：`control/include/pd_controller.hpp`，前置依赖 `angle_utils.hpp`。

3.2 开发任务拆解

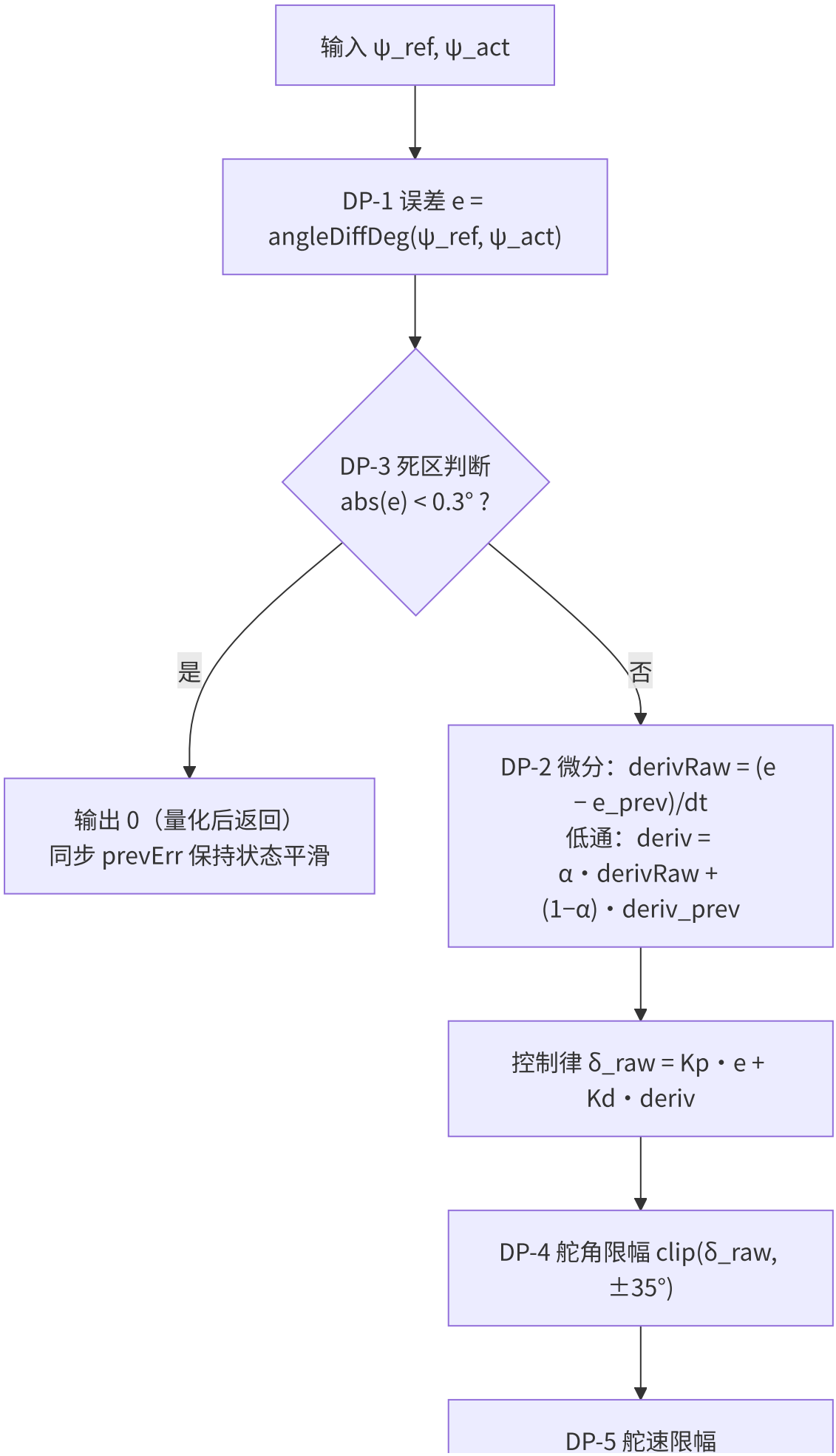
PD 控制器虽然公式简单，但工程实现要点集中在"公式之外"的 6 个开发点，缺一不可：

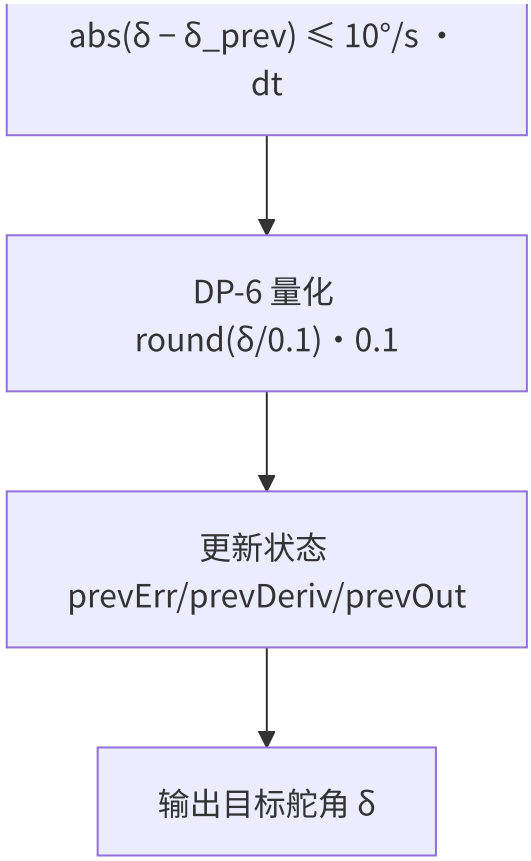
开发点	设计依据	一句话要求
DP-1 角度归一化误差	§3.2	误差必须经 atan2 等价归一化到 (−180°, 180°]
DP-2 微分低通滤波	§3.2/§3.7	差分微分必须带一阶低通，τ = 0.2 s
DP-3 误差死区	§3.3/§3.7	|e| < 0.3° 时输出 0，减少操舵频次

开发点	设计依据	一句话要求
DP-4 舵角限幅	§3.3	输出饱和 ±35°
DP-5 舵速限幅	§3.3	相邻周期输出变化 ⩽ 10°/s × dt
DP-6 输出量化	§3.7	目标舵角量化到 0.1°（12-bit 舵角分辨率）

3.3 处理链与执行顺序

执行顺序是本模块最重要的实现约束：死区判断最先（直接短路返回），量化最后；舵速限幅必须在舵角限幅之后，否则限幅截断会破坏速率约束。


输入 ψ_ref, ψ_act
DP-1 误差 e = angleDiffDeg(ψ_ref, ψ_act)
DP-3 死区判断 abs(e) < 0.3° ?
是
否
输出 0（量化后返回）
同步 prevErr 保持状态平滑
DP-2 微分：derivRaw = (e − e_prev)/dt
低通：deriv = α・derivRaw + (1−α)・deriv_prev
控制律 δ_raw = Kp・e + Kd・deriv
DP-4 舵角限幅 clip(δ_raw, ±35°)
DP-5 舵速限幅



对应实现即 `PdController::update()` 的处理链：

```
    double update(double headingRef, double headingAct) {
        // §3.2 角度归一化的航向误差
        const double e = angleDiffDeg(headingRef, headingAct);

        // §3.7 误差死区
        if (std::abs(e) < p_.deadzone) {
            // ... 死区短路返回 ...
        }
        // ... 微分低通 → PD → 舵角限幅 → 舵速限幅 → 量化 ...
    }
```

3.4 各开发点实现细节

DP-1 角度归一化误差

$$e = \operatorname{atan2}(\sin(\psi_{\text{ref}} - \psi),\ \cos(\psi_{\text{ref}} - \psi))$$

工程实现不必调用三角函数，用 while 折叠等价实现（`angle_utils.hpp` 的 `normalizeAngleDeg`）：结果落在 **(−180°, 180°]**，注意区间是左开右闭——`normalizeAngleDeg(180)` 必须返回 +180 而不是 −180，边界条件用 `a <= -180` 判断实现。

典型错误示例：ψ_ref = 1°、ψ_act = 359°，直接相减得 −358°，归一化后应为 **+2°**（右转 2° 而不是左转 358°）。

DP-2 微分低通滤波

$$\dot{e}_k = \alpha \cdot \frac{e_k - e_{k-1}}{\Delta t} + (1-\alpha)\cdot \dot{e}_{k-1}, \qquad \alpha = \frac{1}{1+\tau/\Delta t}$$

- 取 τ = 0.2 s、dt = 0.02 s 时 α ≈ 0.0909，即新样本权重约 9%，等效截止频率约 0.8 Hz，可压制 AHRS/GNSS 高频噪声。
- **首拍处理**：第一拍没有 e_{k−1}，若直接差分会产生一次 e/dt 量级的微分冲击（e = 30° 时冲击达 1500°/s）。实现上用 `first_` 标志令首拍 `derivRaw = 0`。
- **微分对象的选择**：本实现对误差 e 微分。若 ψ_ref 会被上层频繁阶跃修改（如机动序列切段），对 e 微分会引入"设定值踢"（derivative kick）；缓解手段是切换目标航向时调用 `reset()`，或后续版本改为对测量值 −ψ 微分。

DP-3 误差死区

- 死区内输出 0 而不是"保持上一输出"，语义为"回中"；死区内仍要更新 `prevErr_`，否则退出死区的第一拍会把死区期间积累的误差变化一次性算进微分。
- **死区边界抖动**：e 在 0.3° 附近来回穿越时输出在 0 与非 0 间跳变。当前实现依靠舵速限幅（DP-5）天然抑制跳变幅度；若海试发现操舵频次超标（> 6 次/分钟），加迟滞（如 0.3° 进入 / 0.5° 退出）。

DP-4 / DP-5 双重限幅

$$\delta = \mathrm{clip}(\delta_{\mathrm{raw}},\ \pm\delta_{\max}), \qquad |\delta_k - \delta_{k-1}| \le \dot{\delta}_{\max}\cdot\Delta t$$

- 顺序固定为 **先幅值后速率**。dt = 0.02 s 时单周期最大变化量 10 × 0.02 = 0.2°，即从 0 打到 35° 满舵至少需要 3.5 s，与舵机物理能力一致。
- 舵速限幅基于 `prevOut_` 递推，因此 `reset()` 后第一拍从 0 开始爬升；上电/切入自动模式时舵不在中位的问题已解决（P11 整改已落地）：切入 AUTO_HEADING 的首拍以实际舵角反馈调 `initBaseline()` 初始化舵速限幅基准，舵令不再从 0° 爬升（§9.8 EnterAutoHeadingSeedsRudderBaseline 回归用例）。

DP-6 输出量化

`round(v / 0.1) * 0.1`。量化必须放在处理链最后，否则量化误差会被后级放大。量化本身也兼具消除极小抖动输出的作用。

3.5 参数与状态

类别	成员	说明
参数（`Params`）	Kp, Kd, dMax, ddMax, deadzone, tau, quant, dt	全部带默认值，可整组替换
内部状态	prevErr_, prevDeriv_, prevOut_, first_	`reset()` 统一清零

注意： `setParams()` 内部调用 `reset()`，这意味着**每次改参数都会清空微分历史与舵速限幅基准**。增益调度按周期改参数时必须避开该行为（详见 §9.4 与 §10.5-P1）。

3.6 测试要点

前置依赖 `angle_utils` 的用例（对应 §10.1 汇总表 angle_utils 行）：

用例	输入	预期
正边界（左开右闭）	normalizeAngleDeg(180)	+180（不是 −180）
负边界	normalizeAngleDeg(−180)	+180
正向折叠	normalizeAngleDeg(540)	+180
负向折叠	normalizeAngleDeg(−541)	+179
跨零差值	angleDiffDeg(1, 359)	+2
反向跨零差值	angleDiffDeg(359, 1)	−2

PD 控制器用例：

用例	输入	预期
跨零误差	ref=1, act=359	e = +2，输出为正（右舵）
死区	ref=100, act=99.8	输出 0
舵角饱和	Kp=10, e=170°	输出被限幅（受舵速限幅逐拍爬升，稳态 = ±35°）
舵速限幅	阶跃 e：0 → 30°	相邻输出差 ⩽ 0.2°（dt=0.02）
首拍无冲击	首次 update，e=30°	输出 ≈ Kp · e 限幅限速后的值，无微分尖峰
量化	任意	输出为 0.1° 整数倍

用例	输入	预期
低通有效性	e 叠加 5 Hz 噪声	输出微分分量显著衰减（> 80%）

---

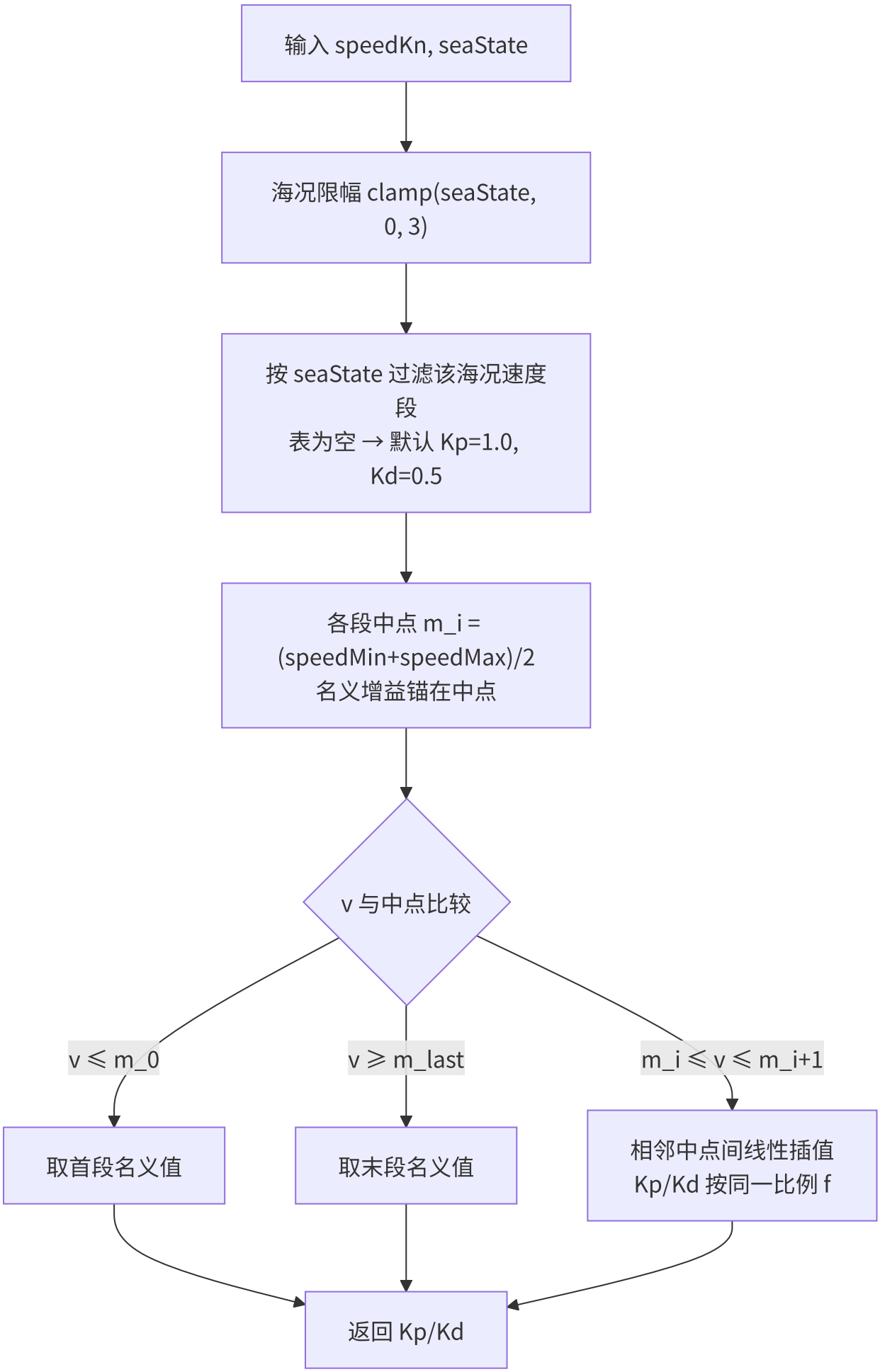
4. 增益调度开发

4.1 设计依据

设计文档 §3.5（PD 参数整定示例表）、§3.6（增益调度）、§3.10.4（按船速分段的 ζ*/ωn* 表）。实现文件：`control/include/gain_schedule.hpp`。

4.2 算法拆解

增益调度是一张 **船速 × 海况 → (Kp, Kd)** 的二维查表，速度维按段中点锚定线性插值（P5 整改，消除段切换跳变）。物理动因：船速越高、海况越差，等效模型增益 K 越大且噪声越强，需要越保守的控制增益。


输入 speedKn, seaState
海况限幅 clamp(seaState, 0, 3)
按 seaState 过滤该海况速度段
表为空 → 默认 Kp=1.0, Kd=0.5
各段中点 m_i = (speedMin+speedMax)/2
名义增益锚在中点
v 与中点比较
v ⩽ m_0
v ⩾ m_last
m_i ⩽ v ⩽ m_i+1
取首段名义值
取末段名义值
相邻中点间线性插值 Kp/Kd 按同一比例 f
返回 Kp/Kd

4.3 调度表数据结构与默认内容

```
struct GainPoint {
    double speedMin, speedMax;  // [speedMin, speedMax) kn
    int    seaState;            // 0..3
    double Kp, Kd;
};
```

默认表共 14 条记录（4 个速度段 × 海况分组），与设计文档 §3.5 示例表和 §3.10.4 分段一致：

船速区间 (kn)	海况 0~1	海况 2~3
[0, 4)	Kp=2.0, Kd=0.8	—（同 0~1，低速不区分）
[4, 10)	Kp=1.5, Kd=0.6	Kp=1.0, Kd=0.4
[10, 18)	Kp=1.2, Kd=0.5	Kp=0.9, Kd=0.4
[18, 25)	Kp=1.0, Kd=0.4	Kp=0.7, Kd=0.3

4.4 实现细节与易错点

1. **段中点锚定插值（P5 整改后语义）**：名义增益锚在每段中点 m_i = (speedMin+speedMax)/2；v 落在相邻中点 [m_i, m_{i+1}] 间时按同一比例 f 对 Kp/Kd 线性插值——中点处恰取名义值（NominalAtMidpoint），跨中点交界连续（InterpolationContinuityAtBoundary）。段边界（如 10.0 kn）只是普通插值点，不再是跳变点。
2. **速度超出中点覆盖范围**：v ≤ 首段中点（2 kn）取首段名义值，v ≥ 末段中点（21.5 kn）取末段名义值——等价于"两端外推为常值"，保证高速仍返回最保守增益而不是失败。
3. **海况输入防御**：外部海况估计可能给出越界值，入口 `clamp(0, 3)`。
4. **段切换跳变已消除（§10.5-P5 已关闭，P5 整改已落地）**：旧实现为阶跃查表，船速穿越段边界时 Kp 从 1.5 跳到 1.2，经 PD 输出会产生一次小的舵角阶跃；现按当时的方向一（速度维相邻段线性插值）实现，跳变源头消除，方向二（查表结果加一阶低通 / 斜坡限速）未采用亦不需要。
5. **自定义表替换**：整表可由标定结果替换（配合 `param_store` 的区块 D），替换时必须校验"每个海况分组的速度段完整覆盖 [0, 25) 且无重叠"，建议提供 `validate()` 工具函数。

4.5 增益切换与 PD 的衔接

查表结果每周期写入 PD 参数。由于 `PdController::setParams()` 会 `reset()` 内部状态（§3.5 注意事项），衔接方式必须是下列之一：

- **方案 A（推荐）**：给 `PdController` 增加 `setGains(Kp, Kd)` 轻量接口，只改增益不清状态；
- 方案 B：仅当增益变化超过阈值（如 1%）时才调用 `setParams`，并接受一次状态清零。

已按方案 A 落地（P1 整改，§10.5-P1 已关闭）：`PdController` 提供 `setGains(Kp, Kd)` 与 `setDt(dt)` 轻量接口，只改增益/采样周期、不清状态；`headingControlStep` 每拍经它们转发调参器输出增益，主循环不再调 `setParams`。

4.6 测试要点

P5 整改后现状 7 用例（test_gain_schedule.cpp）：

用例	输入	预期
NominalAtMidpoint	v=7（[4,10) 段中点）, sea=0	名义值 Kp=1.5, Kd=0.6
InterpolationContinuityAtBoundary	v=14（相邻中点交界）±1e-9, sea=0	中点处 = 名义 Kp=1.2，两侧极限连续
HighSeaState	v=7, sea=2	Kp=1.0, Kd=0.4
SeaStateClamp	v=7, sea=9	与 sea=3 结果逐位一致
SpeedAboveMax	v=30, sea=0	末段名义值 Kp=1.0, Kd=0.4
SpeedBelowMin	v=0, sea=0	首段名义值 Kp=2.0, Kd=0.8
NoCrashOnNegativeSpeed	v=−1, sea=0	取首段名义值，不得崩溃

---

5. Nomoto 模型辨识开发

5.1 设计依据

设计文档 §3.4（Nomoto 一阶模型）、§3.9.4（Z 形试验离线辨识）、§3.10.3（RLS 在线辨识）。实现文件：`control/include/nomoto_identifier.hpp`。

5.2 模型离散化推导

连续模型 $T\dot{r} + r = K\delta$，用后向差分 $\dot{r} \approx (r_k - r_{k-1})/\Delta t$ 离散化：

$$r_k = K\delta_k - \frac{T}{\Delta t}(r_k - r_{k-1})$$

整理为线性回归 $y_k = \phi_k^T \theta$：

$$\phi_k = \left[\delta_k,\ -\frac{r_k - r_{k-1}}{\Delta t}\right]^T, \quad y_k = r_k, \quad \theta = [K,\ T]^T$$

该形式的价值在于：**K、T 同时以线性参数出现**，离线最小二乘与在线 RLS 共用同一套回归量构造代码。

5.3 两条辨识路线

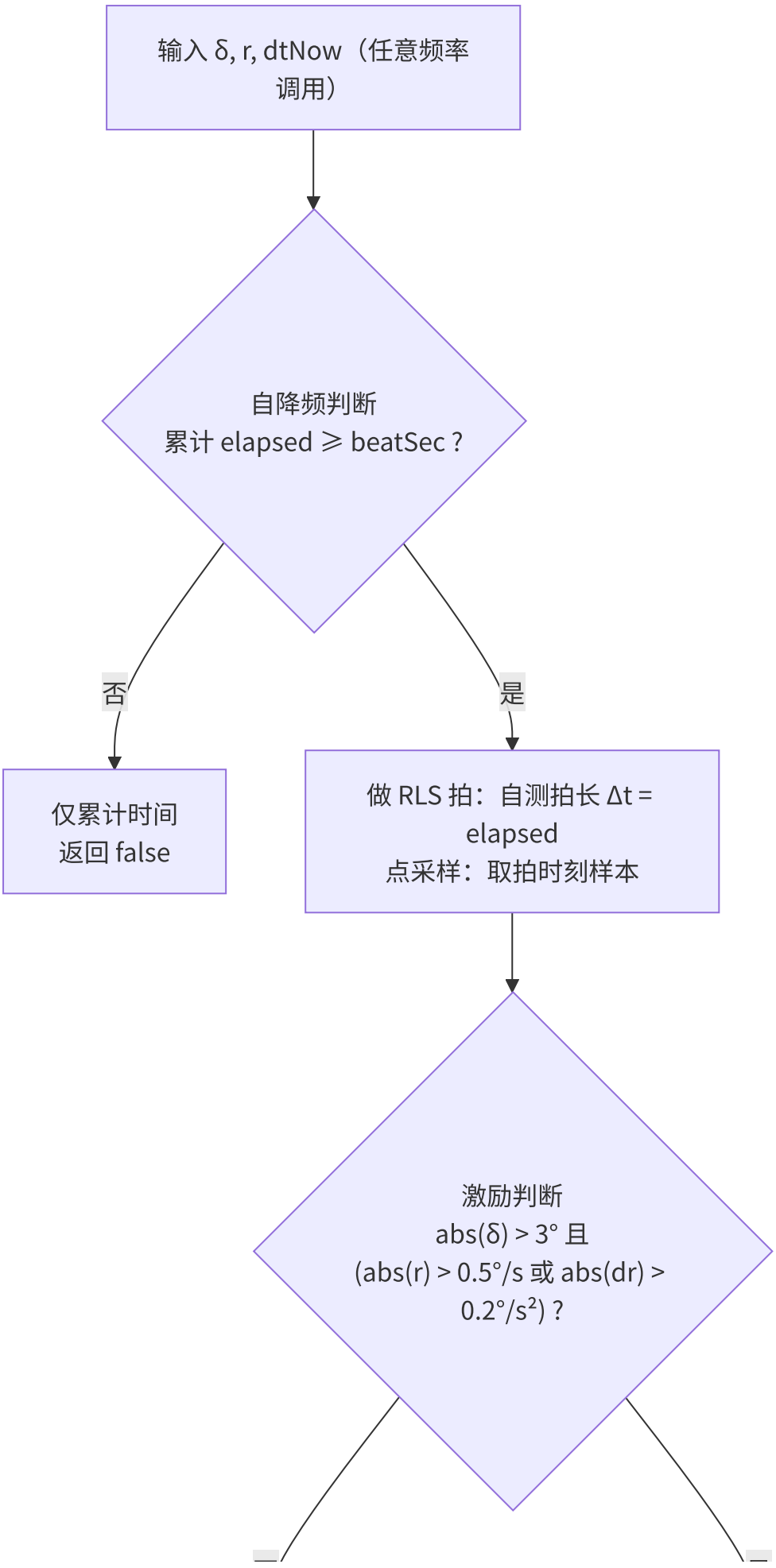
项	路线 A：离线批量 LS（§3.9.4）	路线 B：在线 RLS（§3.10.3）
使用场景	装船标定、海试 Z 形试验	运行中持续跟踪 K/T 慢变
激励来源	人工 Z 形试验（±10°/10°，3~5 循环）	正常操舵自然激励
算法	$\hat{\theta} = (\Phi^T\Phi)^{-1}\Phi^T y$	带遗忘因子 λ=0.98 的递推 LS
载体	PC 离线脚本（M2 交付的辨识工具）	ECU 内 `NomotoIdentifier`
验收	拟合残差 RMS < 5% · r_max	仿真数据上收敛误差 < 5%

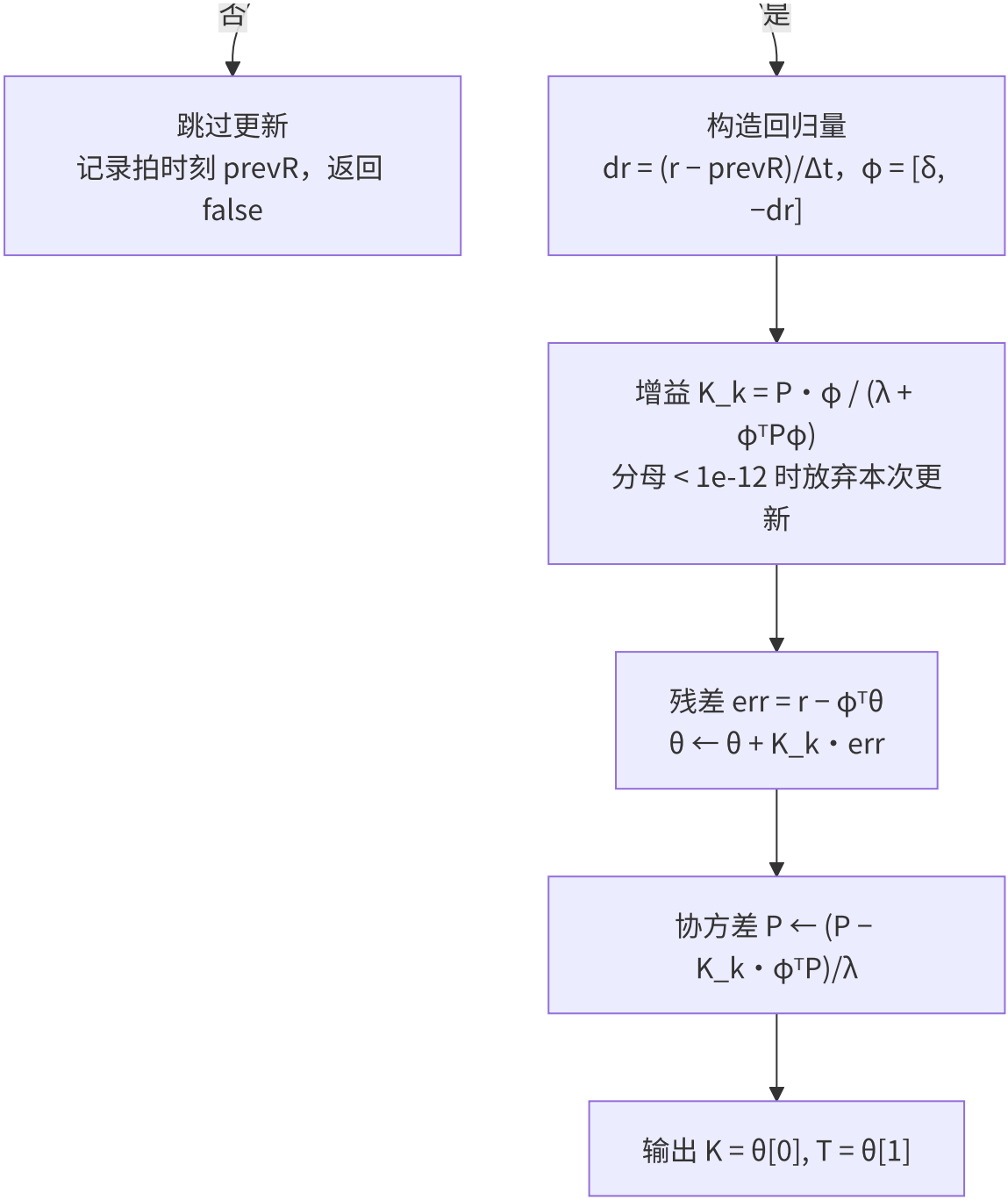
两条路线共享回归量构造，离线工具建议直接复用 `NomotoIdentifier`（λ 置 1.0、P0 足够大即可等效批量 LS 的递推形式）。

5.4 RLS 递推实现

$$K_k = \frac{P_{k-1}\phi_k}{\lambda + \phi_k^T P_{k-1}\phi_k}, \qquad \hat{\theta}_k = \hat{\theta}_{k-1} + K_k\left(y_k - \phi_k^T\hat{\theta}_{k-1}\right), \qquad P_k = \frac{1}{\lambda}\left(P_{k-1} - K_k\phi_k^T P_{k-1}\right)$$

R1 深化（自降频接口）：`update(δ, r, dtNow)` 可由调用方按任意频率调用，模块内部累计满 `Params.beatSec`（默认 1.0 s）才做一次 RLS 拍；差分 Δt 由模块自测（两次 RLS 拍的真实间隔），与调用频率无关。旧设计把"50 Hz × 1 s"的降频约定放在调用方（计数器 + 外部改写 `Params.dt` 为 1.0），调用频率一变差分即静默出错——该语义陷阱已随 `Params.dt` 字段一并删除。


输入 δ, r, dtNow（任意频率调用）
自降频判断
累计 elapsed ≥ beatSec ?
否
是
仅累计时间
返回 false
做 RLS 拍：自测拍长 Δt = elapsed
点采样：取拍时刻样本
激励判断
abs(δ) > 3° 且
(abs(r) > 0.5°/s 或 abs(dr) > 0.2°/s²) ?



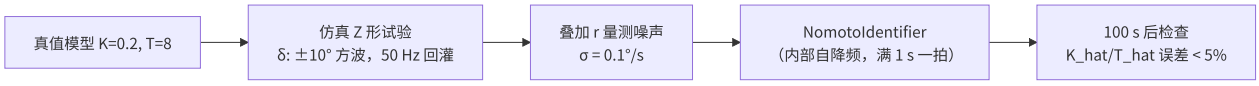
5.5 实现细节与易错点

1. **激励门限是数值稳定的生命线**（$|\delta| > 3°$ 且 $|r| > 0.5°/s$）。直航时 δ、r 均近零，ϕ ≈ 0，若仍更新则 P 阵按 1/λ 每步放大 2%（"协方差爆炸"），一旦重新出现激励，θ 会剧烈跳变。无激励时**必须完全跳过 P 与 θ 的更新**。
2. **首拍差分**：第一个 RLS 拍无 r_{k−1}，令 dr = 0。R1 自降频后 `prevR_` 只在 RLS 拍时刻更新（非拍调用不写），差分恒跨"两次 RLS 拍"，Δt 取两拍真实间隔——长时间未激励后重新进入激励段时差分仍连续，无需额外处理。另注意被拒绝的拍（NaN/超限/非法 dtNow）不写 `prevR_`，否则毛刺会成为下一拍差分锚点（P11）。

3. **分母保护**：`λ + φᵀPφ` 理论上恒正，但浮点异常输入（NaN/Inf 传感器值）可能破坏该性质，实现中做了 `< 1e-12` 防御；工程上还应在入口对 δ、r 做范围检查（|δ| ⩽ 35°、|r| ⩽ 30°/s，超限丢弃）。
4. **物理合理性检查**（已实装，P6 整改）：`valid()` 门限 K ∈ (0, 2]（1/s 量级）、T ∈ [1, 60] s（非有限值一并拒绝）；越界说明数据质量差——辨识快照 `valid=false` 时调参器（周期评估与 `establishBaseline`）不会把该结果送入层 2 极点配置（§ 10.5-P6 已关闭）。
5. **调用节拍（R1 自降频）**：设计文档 §3.10.3 规定辨识每 **1 s** 更新一次（降低 ECU 负担并保证差分信噪比）。R1 深化后该节拍由模块自维护：主循环按 50 Hz 直接调 `update(δ, r, dtNow)`，内部累计满 `Params.beatSec`（默认 1.0 s）才做一次 RLS 拍，差分 Δt 自测（两次 RLS 拍的真实间隔），调用频率变化不影响正确性；`Params.dt` 字段已删除，不再需要调用方改写（§ 10.5-P3 由 R1 根治）。
6. **单位一致性**：全链路用度（deg、deg/s），则辨识出的 K 单位为 (deg/s)/deg = 1/s，与弧度制数值相同，可直接用于极点配置公式。

5.6 收敛性单测方案

用已知参数的 Nomoto 模型生成仿真数据回灌，是 M2 准出的核心用例：



用例	场景	预期
无噪收敛	干净 Z 形数据	30 个有效样本内误差 < 1%
带噪收敛	σ=0.1°/s 噪声	100 s 内误差 < 5%
自降频任意调用率（SelfDecimatesAtAnyCallRate，R1 新增）	同一 Z 形数据流按 50 Hz 回灌（生产调用形态），激励按 RLS 拍翻转	内部满 beatSec=1 s 才做一拍，Δt 自测正确，收敛误差与 1 s 直调等价
无激励保护	全程 δ=0	θ 保持初值，P 不增长，`update` 恒返回 false
参数慢变跟踪	K 在 200 s 内线性 +50%	λ=0.98 下 K_hat 跟踪滞后 < 30 s
坏数据防御	混入 NaN/超限样本	不污染 θ，无 NaN 扩散

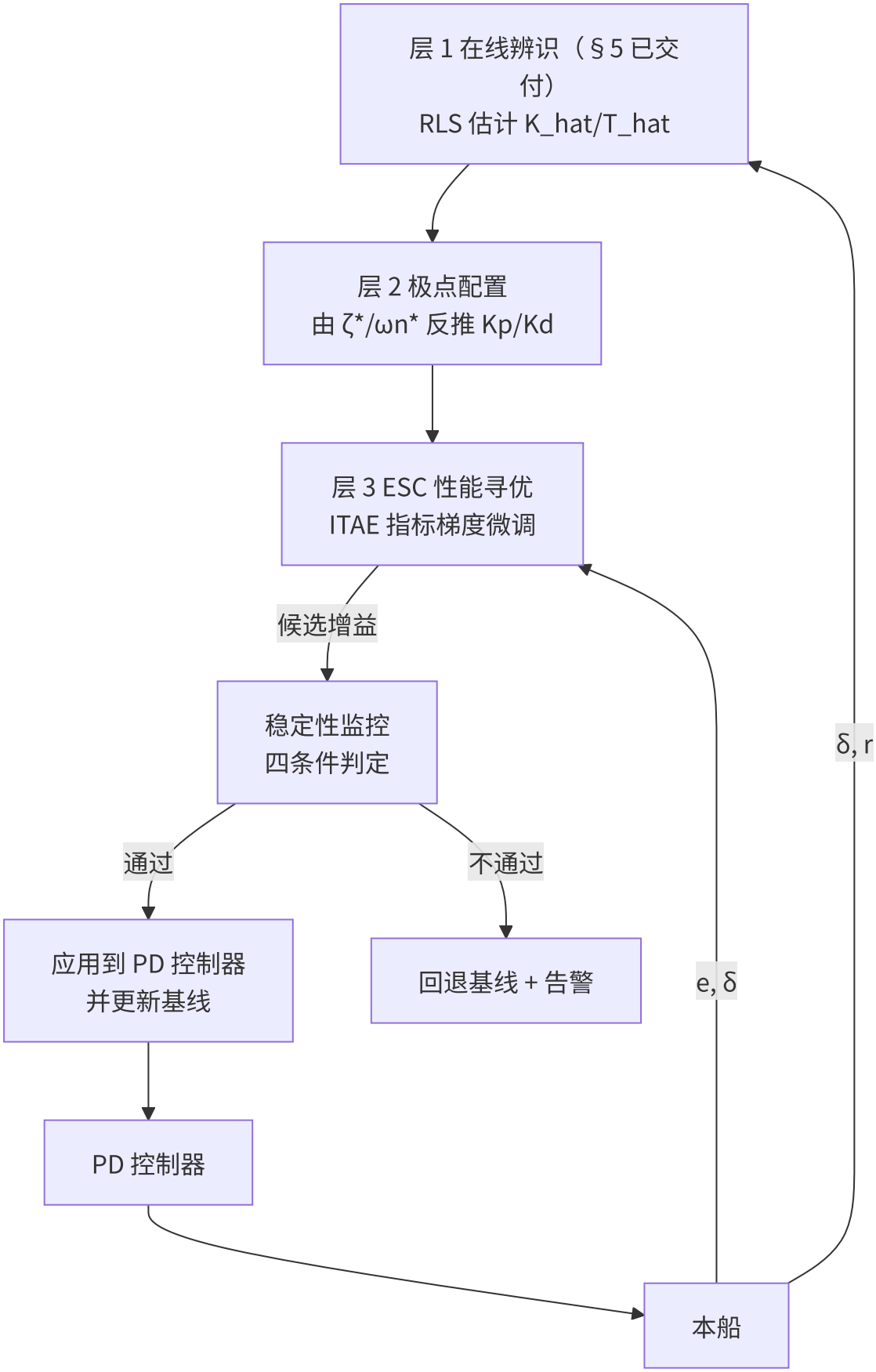
---

6. 控制参数自动优化（AutoTuner）开发

6.1 设计依据

设计文档 §3.10 全节。实现文件：`control/include/auto_tuner.hpp`，依赖 `nomoto_identifier.hpp`（层 1）。

6.2 三层架构与开发次序



开发次序即 L2 → L3 → 监控（层 1 在 M2 已完成）：极点配置是纯函数最简单；ESC 依赖性能指标窗口；监控最后接线，因为它要拦截前两者的全部输出。

接口现状（R1 深化）：三层机制的编排（节拍计数、窗口结算、速度门控、增益所有权、漂移重基线、振荡门控）已全部收进 AutoTuner 内部，对外动作只剩 3 个：

接口	语义
`tick(const TunerInput&) → GainCommand`	每拍单入口。内部顺序：速度门控（`enabled_ = speedKn ⩽ highSpeedDeactivateKn`）→ 无基线时跟随调度值 → 周期辨识评估（P12 漂移检查）→ 参考稳定门控 + 性能样本累积（含因果解调）→ ESC 窗口结算（稳定门 + 回退）→ 扰动注入 → 返回本拍增益（`GainCommand` 已含 ESC 载波）
`establishBaseline(const IdSnapshot&) → bool`	一次性评估并建基线（P14 辨识试验收官 / 人工标定注入）；不做漂移检查、振荡条件以 0 把关、恒重锚 ESC；与 tick 共享私有 `applyEstimate_()`
`suspend()`	唯一模式生命周期钩子：停用 + 清测量状态（窗口/解调/HPF/梯度），保留基线

输入输出结构（namespace `ar`）：`TunerInput{headingRefDeg, headingDeg, rudderDeg, speedKn, dt, schedKp, schedKd, id}`；`IdSnapshot{excited, valid, K, T}` 为辨识快照（辨识器保持独立模块，由调用方每拍喂数据并组装快照传入）。只读查询：`enabled()`、`baselineEstablished()`、`currentGains(Kp&, Kd&)`（去载波投影，观测增益所有权用）、`computeJ()`、`oscDetector()`；`polePlacement`（公有静态，公式全仓库唯一出处）与 `stabilityCheck`（公有纯函数）维持公有。原公有机制 `accumulate` / `escDither` / `escStep` / `revert` / `rebaseEsc` / `setBaseline` 已私有化，一律经 tick/establishBaseline 驱动；ESC 窗口结算为私有 `settleWindow_()`（原 `AutopilotCore::escTrigger` 内收）。

6.3 层 2：极点配置

由期望闭环 ζ*、ωn* 与辨识值反推（设计文档 §3.10.4）：

$$K_p = \frac{(\omega_n^*)^2\,\hat{T}}{\hat{K}}, \qquad K_d = \frac{2\zeta^*\omega_n^*\,\hat{T} - 1}{\hat{K}}$$

实现为静态纯函数 `AutoTuner::polePlacement(K, T, zeta, wn, Kp, Kd)`，便于独立单测。

实现细节：

1. **除零保护**（已关闭，P6 整改已落地）：|K| < 1e-6 或 |T| < 1e-6（含非有限值）时 `polePlacement` 返回 false、不产生增益——Kp=0 意味着开环，直接送给 PD 是危险的；调用方（`applyEstimate_()`）在 false 时保持原增益。

2. **Kd 正性约束**：公式在 $2\zeta^*\omega_n^*\hat{T} < 1$ 时给出负 Kd（例如 ζ*=0.85、ωn*=0.2 时要求 T > 2.94 s）。小 T 快响应船本就不需要微分补偿，但负 Kd 会反向放大振荡，**必须 clamp 到 Kd ⩾ 0** 并记录告警。
3. *ζ/ωn* 按船速分段**（设计文档 §3.10.4 表）：< 4 kn 取 (0.9, 0.15)、4~10 kn 取 (0.85, 0.20)、10~18 kn 取 (0.85, 0.25)、18~25 kn 取 (0.90, 0.20)。当前 Params 只有单组 zeta/wn，分段逻辑由上层在船速变化 > 2 kn 时改写 Params 实现（现状：单组缺省 ζ=0.85/ωn=0.20，船速分段未实装——代码中尚无改写 Params 的上层逻辑，列为后续项）。
4. **验证公式**（与 §3.8 一致，用于监控反算）：

$$\omega_n = \sqrt{\frac{K_p\hat{K}}{\hat{T}}}, \qquad \zeta = \frac{1 + K_d\hat{K}}{2\sqrt{\hat{T}\,K_p\hat{K}}}$$

将 polePlacement 的输出代回，应精确还原 ζ*、ωn*（单测断言，容差 1e-9）。

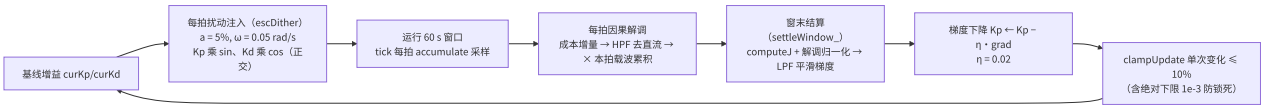
6.4 层 3：ESC 极值搜索

6.4.1 性能指标（ITAE + 控制能耗）

$$J = \int_0^{T_w} \Big(t\,|e(t)| + w_u|\delta(t)| + w_{\dot{u}}|\dot{\delta}(t)| \Big)\, dt$$

- 评估窗口 T_w = 60 s，w_u = 0.01、w_du = 0.005（Params 默认值）。
- 实现为三个定长环形数组（`windowErr_/windowDelta_/windowDDelta_`，P2 整改：无堆分配，容量上限 T_w/dt = 3000 样本）；R1 起 `accumulate()` 为私有，由 `tick()` 每拍推入性能样本并同步做 ESC 因果解调累积。
- **注意**：ITAE 的时间权重 t 以窗口起点为零点。该实现语义是"窗口内 ITAE"：窗口非重叠，每次结算后样本清零重计，因此评估两组增益优劣时窗口天然对齐（每个 ESC 步长评估完整的 60 s 窗口）。

6.4.2 ESC 更新流程



实现细节与易错点：

1. **时间尺度分离**是 ESC 收敛的前提：扰动周期 2π/0.05 ≈ 126 s ≫ 闭环调节时间（⩽ 30 s）≫ 控制周期 20 ms。窗口结算（`escStep`）只能按窗口周期（60 s）执行一次。R1 深化后该节拍

内收：`tick()` 按 `windowSec/dt` 换算窗口拍数并自维护 `winCounter_`，满窗才调私有 `settleWindow_()` ——主循环无需也不应再有外部 60 s 调度器（原 core 的 `escTrigger` /外部触发编排已删除）。

2. **相位错开 90°**：Kp 用 sin(φ)、Kd 用 cos(φ)，两通道扰动正交，解调时互不耦合。
3. **标准 ESC 结构（P11 已落地）**：扰动真实注入增益（每拍叠加 a・sin/cos 载波），每拍做因果解调——成本增量经按控制周期 dt 设计的一阶 HPF 去直流后乘以本拍载波累积；窗末归一化为梯度（$\hat{g} = 2 \cdot \Sigma(y \cdot 载波)/(a \cdot N)$），再经按 windowSec 计算的 LPF 平滑后积分。旧"简化解调"（dJ×sin 相关、增益窗内恒定）的梯度没有因果来源、被衰减 2~3 个量级，已废弃。
4. **禁用状态短路**：`enabled_ == false` 时 `escDither` / `escStep` 直接返回，增益冻结；`enabled_` 由 `tick()` 每拍按速度门控自维护（§6.4.3）。

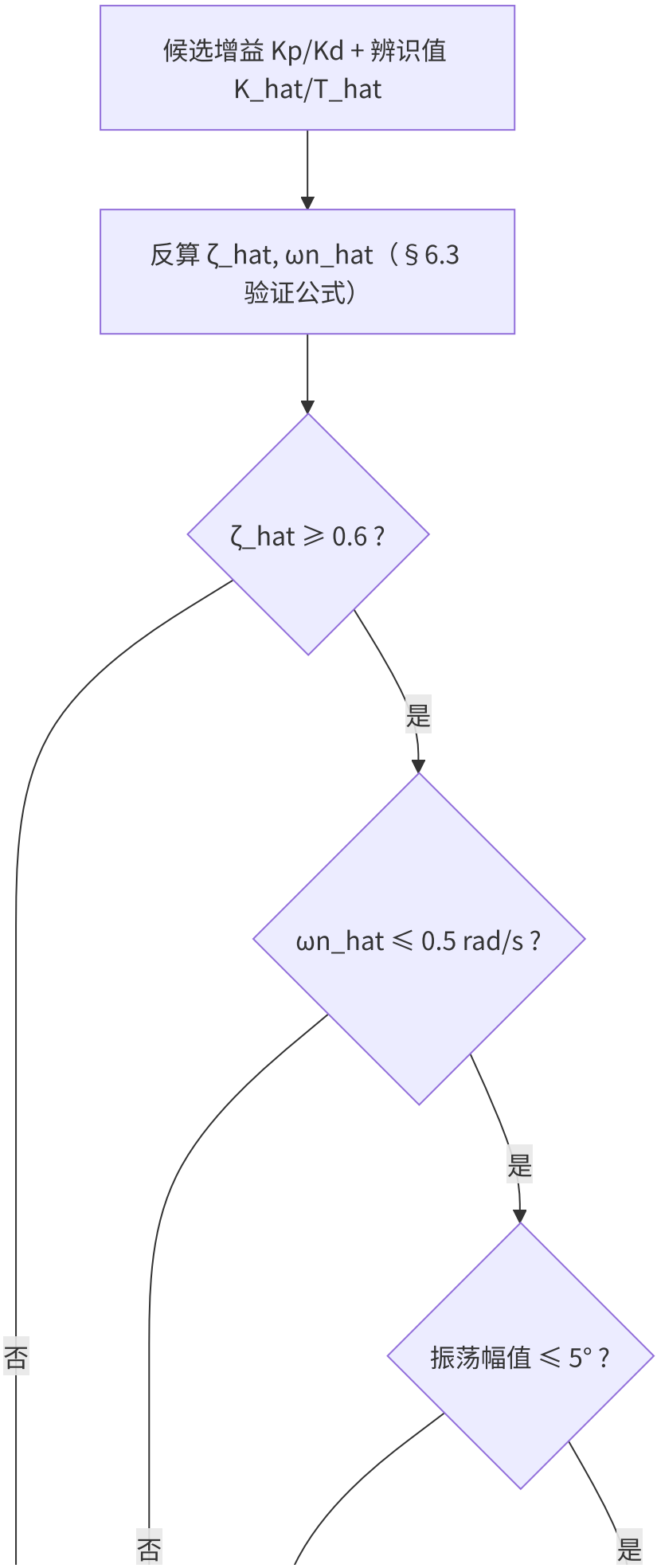
6.4.3 触发与去激活

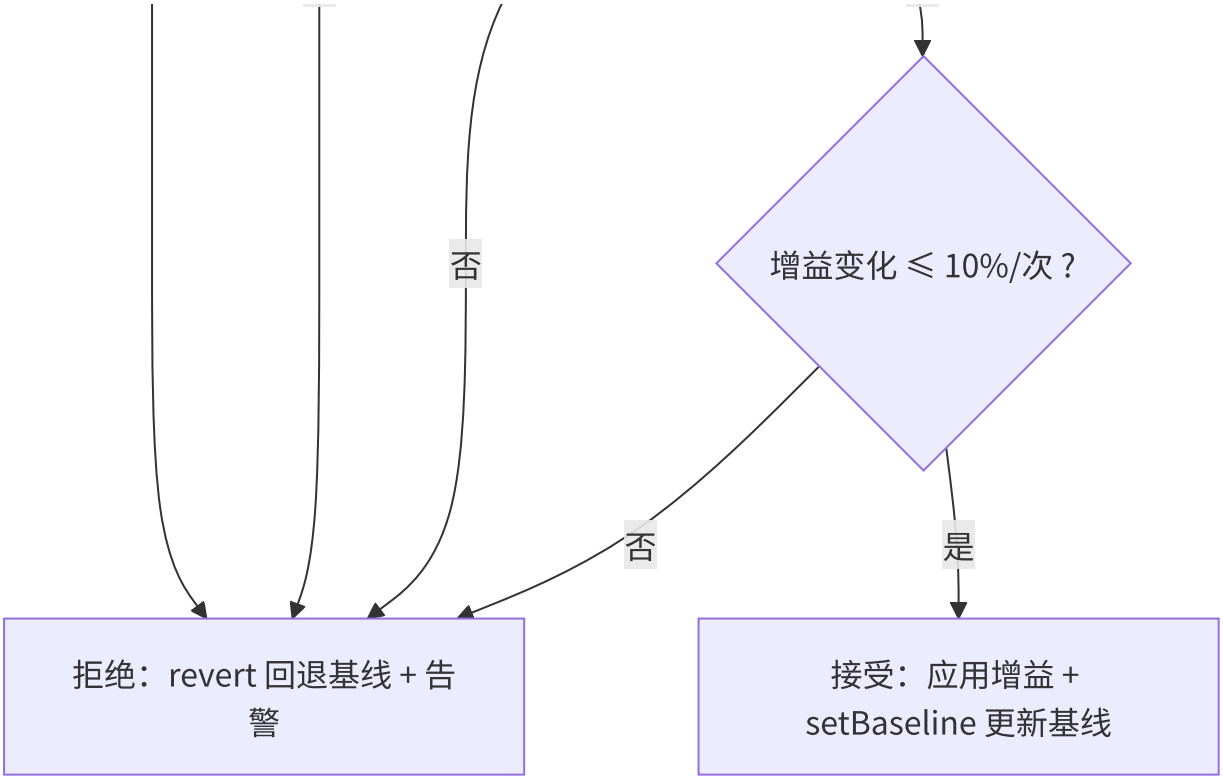
设计文档 §3.10.6 的触发矩阵在 R1 深化后的实现位置——编排已内收进 AutoTuner，上层（`autopilot_core`）不再持有任何节拍计数器或增益副本：

触发条件	动作	实现位置（R1 现状）
船速变化 > 2 kn	重算层 2	无需专门检测：`tick()` 每拍做周期辨识评估，K/T 估计相对基线漂移超 ±25%（`kReBaselineTol`）时经私有 `applyEstimate_()` 自动重配置并重锚 ESC（P12）
稳态误差 > 2° 持续 30 s	启动层 2 + 层 3	当前无独立触发：基线建立后 ESC 持续在线微调（每拍扰动注入 + 60 s 窗口结算）
振荡检测（超调 > 15%）	触发层 2 重算	振荡幅值由 OscillationDetector 实测接入稳定门 C3（P4），窗口结算不通过即回退基线；参考稳定满 30 s（`kOscRefSettleSec`）才喂入检测器，机动期间的误差摆动不参与（P12）
船速 > 15 kn	默认去激活	`tick()` 内速度门控：`enabled_ = (speedKn ≤ highSpeedDeactivateKn)`
操作员手动指令 / 传感器异常	立即去激活	上层切模式，`AutopilotCore::setMode` 调 `suspend()`（清测量状态、保留基线）

6.5 稳定性监控与回退

四条件全满足才接受新增益（设计文档 §3.10.7）：


候选增益 Kp/Kd + 辨识值 K_hat/T_hat
反算 ζ_hat, ωn_hat（§6.3 验证公式）
ζ_hat ⩾ 0.6 ?
是
否
ωn_hat ⩽ 0.5 rad/s ?
是
否
振荡幅值 ⩽ 5° ?
是



实现细节：

R1 深化：上图的稳定门在两条私有路径上生效——周期辨识评估与一次性建基线共享 `applyEstimate_()`（通过才 `setBaseline` 并接管增益）；ESC 窗口结算为 `settleWindow_()`（原 `AutopilotCore::escTrigger` 内收，由 `tick()` 满窗自触发），仅在启用且基线已建立时执行，ESC 更新经 `stabilityCheck`（取当前辨识快照 $\hat{K}/\hat{T}$ 与实测振荡幅值）通过才写入 `curKp_/curKd_`（增益所有权），否则 `revert()` 回退基线。

1. 条件 C4 由 `clampUpdate()` 在更新路径上强制满足（截断而非拒绝），`stabilityCheck()` 实际判定 C1~C3。
2. **振荡幅值检测（P4 已实装，R1 随迁内收）**：`OscillationDetector` 对航向误差 e 做过零结算的峰-峰幅值估计（跨完整周期）；`tick()` 每拍经私有 `accumulate()` 喂入，且仅当航向参考稳定满 30 s（`kOscRefSettleSec`，P12 门控）才喂——参考机动期间的误差摆动是指令响应而非闭环振荡。C3 条件使用实测幅值，不再恒 0。
3. **回退缓存**：设计要求保留最近 3 组验证通过的参数（§3.10.7），当前只有单组 `baselineKp_/baselineKd_`，需扩展为深度 3 的环形缓存，`revert()` 逐级回退。
4. `stabilityCheck` 对 K、T 有 1e-6 防御，K 或 T 无效时直接返回 false（保守拒绝），方向正确。

6.6 测试要点

R1 深化后，状态机制测试一律经公有接口 `tick()` / `establishBaseline()` 驱动——测试与调用方（AutopilotCore）跨同一条 seam，无测试专用后门。现状 17 用例：

用例	场景	预期
PolePlacementProducesTargetZetaWn	K=0.2, T=8, ζ*=0.85, ωn*=0.25	反算 ζ/ωn 还原设定值（1e-6/1e-3）
PolePlacementRejectsDegenerate	K=0 或 T=0	返回 false，不产生增益
PolePlacementClampsNegativeKd	小 ωn 使公式给出负 Kd	Kd clamp 到 ⩾ 0
StabilityCheckAcceptsHealthy	极点配置增益 + 零振荡	四条件通过
StabilityCheckRejectsLowZeta	极小 Kd（低阻尼）	拒绝
StabilityCheckRejectsHighWn	Kp 放大 10 倍使 ωn 超限	拒绝
OscillationDetectorMeasuresAmplitude（P4）	±3° 正弦误差	峰-峰幅值估计 ≈ 6°（±1°）
StabilityCheckRejectsOscillation	振荡幅值 8° > oscMax 5°	拒绝
RingBufferNoHeap（P2）	16 容量环形数组写 100 次	size 封顶 16，按写入顺序遍历正确，无堆分配
OscillationDetectorResetClearsPeaks（P11）	振荡后 reset 再走负半周	历史峰不泄漏进下一窗口，幅值不虚报
EscDitherModulatesGains	建基线后 tick 完整载波周期	输出增益覆盖 ±escAmp 相对调制；无基线时调度增益原样直通（无载波）
WindowSettleUpdateIsClamped（替代 EscStepDoesNotDiverge）	tick 驱动 10 个窗口、误差渐增	逐窗更新受 clampUpdate 约束，或被稳定门拒绝时 revert 回基线
EscTunesKpTowardOptimum	tick + PD + Nomoto 对象整闭环，基线 Kp 故意偏低	若干窗口后 Kp 向改善方向移动且窗口成本 J 下降
EscUpdateUnsticksFromZeroKd（P11）	基线 Kd=0，注入与 Kd 载波负相关的成本	绝对下限 absMinUpdate 解锁更新：Kd > 0 且逐窗有界

用例	场景	预期
SettleRevertsToBaselineWhenEstimateInvalid	梯度非零但快照 valid=false	窗口结算不过稳定门，增益 revert 回基线
SettleAcceptsUpdateWhenEstimateHealthy	同样激励但快照健康	结算被接受，增益按 ESC 梯度上移且有界
RebaselineClearsIntegratorState（替代 RebaseEscClearsIntegratorState，P12）	establishBaseline 二次建基线	旧窗口样本/解调累积作废（computeJ=0），旧梯度不施加到新基线

---

7. 机动序列（ManeuverSequencer）开发

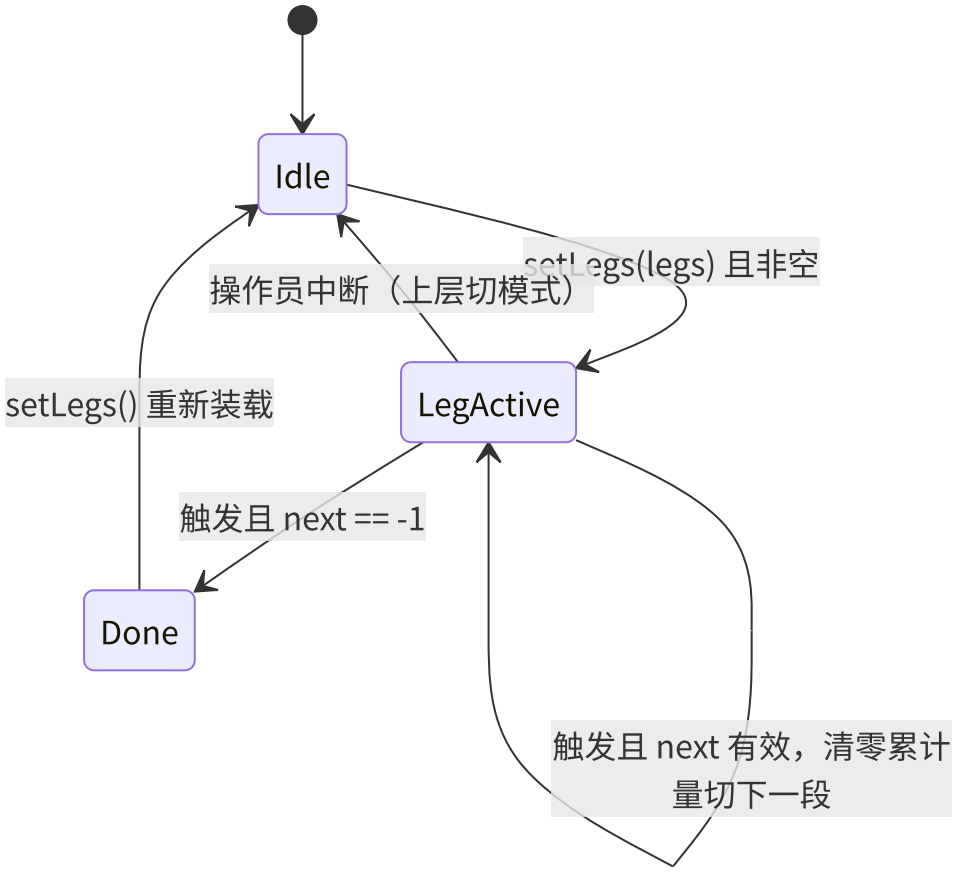
7.1 设计依据

设计文档 §4（仅航向条件下的机动功能方案），尤其 §4.2 通用机动调度框架。实现文件：`control/include/maneuver_sequencer.hpp`，依赖 `angle_utils.hpp`。

7.2 通用框架：Leg 状态机

所有机动统一抽象为 **航段（Leg）序列**，序列器是一个由触发条件驱动的状态机：

```
struct Leg {
    Mode    mode;       // HEADING（PD 跟踪目标航向）/ RUDDER（固定舵角）
    double  target;     // 目标航向(deg) 或 目标舵角(deg)
    Trigger trigger;    // HEADING_REACHED / TIME / RATE / NONE
    double  threshold;  // 触发阈值：累计航向变化(deg) / 时长(s) / 转向率(deg/s)
    int     next;       // 下一段索引，-1 表示序列结束
};
```



每周期 `update(headingAct, rateAct, dt)` 做三件事：**累计状态量 → 判定触发 → 输出当前段指令（并在触发时切段）**。

7.3 触发条件的实现细节（本模块核心难点）

7.3.1 累计航向变化量 cumHeading

`HEADING_REACHED` 触发依赖"**带符号的累计航向变化**"而非当前航向与目标的差值：

$$\text{cumHeading} \leftarrow \text{cumHeading} + \text{angleDiffDeg}(\psi_k,\ \psi_{k-1})$$

- 每拍取相邻两拍航向的**最小带符号差**再累加，因此可正确表达"累计转过 270°、540°"这类超过一圈的语义——这是 Circles（360°）、Williamson（180°）能实现的关键。
- 逐拍差分要求 **相邻两拍航向变化 < 180°**。50 Hz 采样下即使 30°/s 的极限转向率每拍也只有 0.6°，裕量充足；但若上层以低频（如 1 Hz）调用且转向率高，会发生折叠错误——调用周期必须 ⩾ 10 Hz 写入接口约束。
- **首拍基准**：`first_` 标志用当前航向初始化 `lastHeading_`，避免第一拍把 `ψ_act − 0` 误计入累计量。

- **切段清零**：`legElapsed_` 与 `cumHeading_` 在切段时同时清零，每段的阈值都是"本段内"的增量语义。
- 触发判定带**方向敏感**（P12 整改已落地）：`cumHeading · threshold ⩾ threshold²`，即累计量必须沿阈值符号方向达到其幅值才切段——反舵后船舶因惯性顺漂、cum 沿旧方向继续增长时不会把航段"预触发"。阈值带符号（与本段舵向一致），对 Williamson 这类先右后左的机动，反向操舵段阈值为负，语义为"本段内累计净变化量"（见 §7.4.2 的参数推导）。

7.3.2 三类触发对比

Trigger	判定式	适用	注意
HEADING_REACHED	cum · threshold ⩾ threshold²（方向敏感，P12）	转向段	增量语义，切段清零；阈值符号须与本段舵向一致
TIME	legElapsed ⩾ threshold	直航定时段（Cloverleaf/Search）	精度受 dt 粒度限制
RATE	abs(rateAct) ⩽ threshold	"转向率衰减到位"类收尾段	阈值过小会因噪声永不触发，建议 ⩾ 0.5°/s
NONE	永不触发	末段长期保持（如 Williamson Leg3）	由操作员或上层退出

7.4 六种预置机动开发

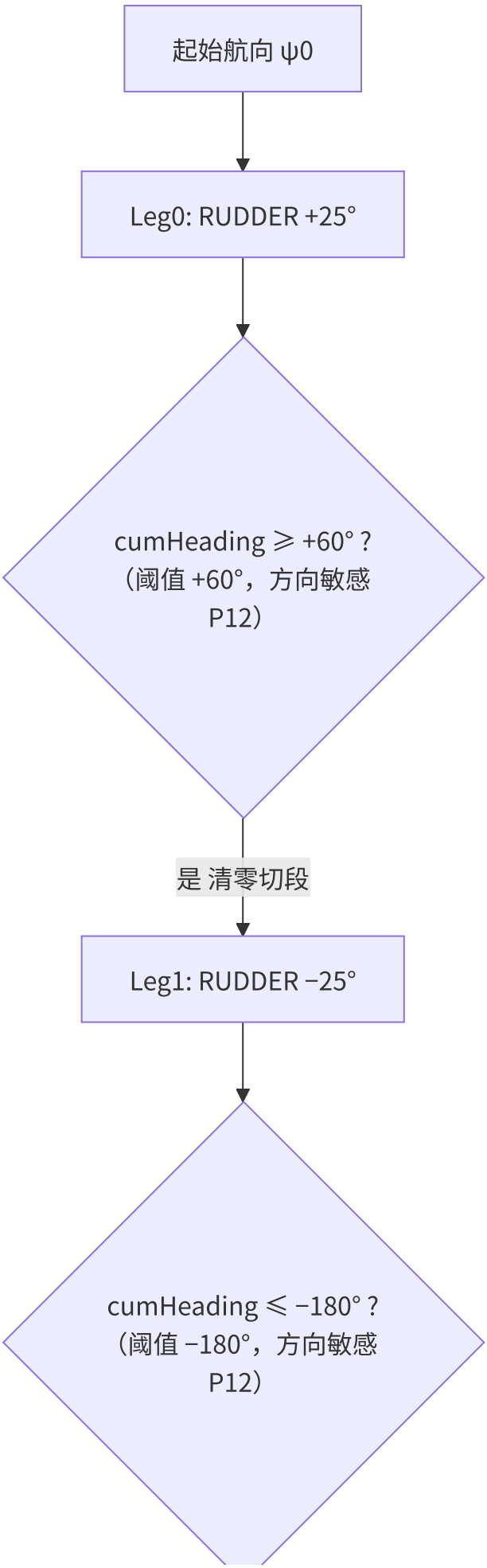
7.4.1 总览

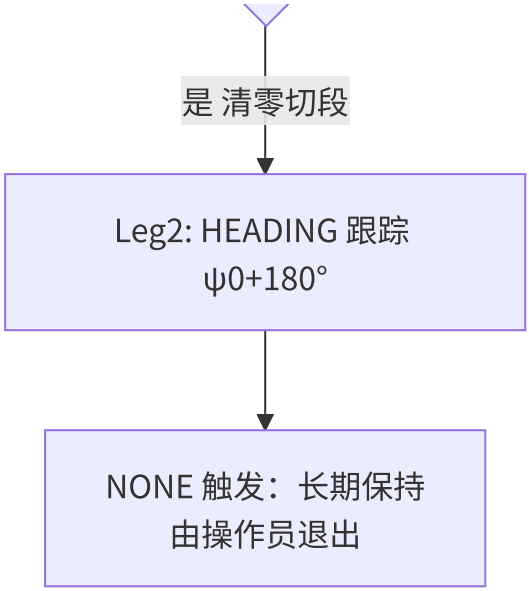
机动	工厂函数	段数	模式组合	可行性（§4.1）
Circles	`circles(dCmd, totalDeg)`	1	RUDDER	完全可行
Williamson	`williamson(psi0, dMax)`	3	RUDDER×2 + HEADING	完全可行
U-Turn	`uTurn(psi0, right)`	2	RUDDER + HEADING	完全可行
Zigzag	`zigzag(psi0, dz, phiz, N, returnToStart)`	2N（+1 保持段）	RUDDER（+HEADING）	完全可行

机动	工厂函数	段数	模式组合	可行性（§4.1）
Cloverleaf	`cloverleaf(psi0, tLeg, dMax)`	8	RUDDER 转向 + HEADING 直航	开环可行（漂移不闭合）
Search	`search(psi0, t0, dt, cycles, dMax)`	2×cycles	HEADING 直航 + RUDDER 转向	开环可行

7.4.2 Williamson Turn（重点机动）

人落水紧急返点机动，三段式（设计文档 §4.4）：


起始航向 ψ0
Leg0: RUDDER +25°
cumHeading ≥ +60° ?
（阈值 +60°，方向敏感
P12）
是 清零切段
Leg1: RUDDER −25°
cumHeading ≤ −180° ?
（阈值 −180°，方向敏感
P12）



参数推导细节：Leg1 的阈值 −180°（带符号 P12，与本段左转舵向一致；判定式 cum · threshold ⩾ threshold²，见 §7.3.1）是"本段内累计净变化 180°"。Leg1 起点航向 ≈ ψ0+60°（右转后），反向左转过程中 cumHeading 从 0 向负方向增长，累计 −180° 时航向 ≈ ψ0−120°；此时切入 Leg2 的 PD 跟踪 ψ0+180°（= ψ0−180°），剩余约 60° 由 PD 闭环收尾。段间衔接的超调由海试标定 dMax（缺省 +25° 而非满舵 35°，留出修正裕量）。

7.4.3 Zigzag（Z 形，兼作辨识激励）

2N 段全 RUDDER 序列，±δ_z 交替、每段以带符号阈值 ±ϕ_z 方向敏感触发（P12：阈值符号随本段舵向，§7.3.1）。工程上它同时是 §5 Z 形试验的**激励发生器**：执行 `zigzag(psi0, 10, 10, 5)` 并同步记录 δ(t)、r(t)，即得到 §3.9.4 要求的辨识数据集。

7.4.4 Circles / U-Turn

- Circles：单段 RUDDER（缺省 +15°），阈值 totalDeg=360°。设计文档 §4.3 的"转向率闭环"方案在当前版本以**恒舵角等效**实现（简化合理：Nomoto 稳态下恒舵角即恒转向率）；若海试发现风流下圆度不佳，升级为转向率外环。
- U-Turn：两段式——RUDDER 恒舵角转向段（转满 180°）+ HEADING 保持段跟踪 ψ0±180°，转向方向由 `right` 参数指定。P7 整改已落地（§10.5-P7 已关闭）：转向段改 RUDDER 后方向由舵角符号强制，不再受旧单段 HEADING 实现的缺陷影响（PD 沿最短路径转向，target 恰好 ±180° 时方向取决于误差符号的数值噪声）。完成判据（设计 §4.5：|e|<1° 持续 2 s）当前依赖 NONE 触发 + 上层判断，需上层实现该判据。

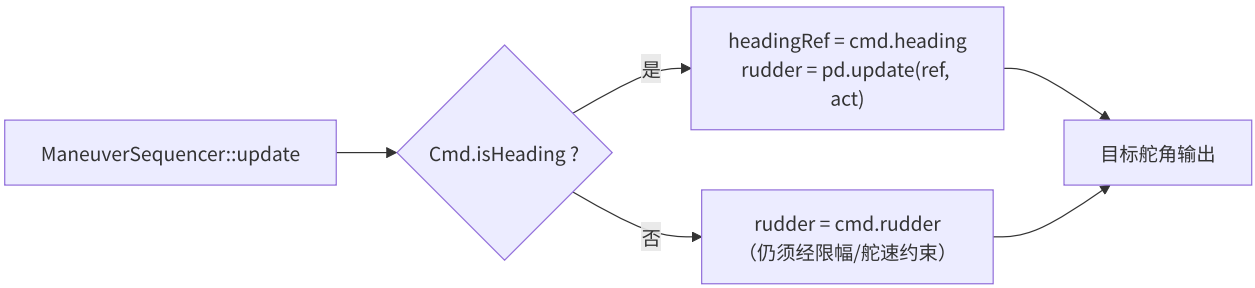
7.4.5 Cloverleaf / Search（开环机动）

两者均为直航段（HEADING + TIME 触发）与转向段（RUDDER 恒舵角）交替，属开环机动（无位置闭合，漂移不可消除，设计 §4.7/§4.8 已声明风险）。

历史缺陷（§10.5-P7，已关闭）：旧实现转向段使用 `Mode::HEADING` + `HEADING_REACHED` 阈值 270°（Cloverleaf）/ 90°（Search），但 PD 跟踪目标航向时走**最短路径**：+270° 的目标等价于 −90°，实际只会累计约 90°，`abs(cumHeading) ⩾ 270°` 永不满足，序列卡死。P7 整改按方案 A 落地：

- 方案 A（已采用）：转向段改为 RUDDER 恒舵角（同 Williamson），达到累计角度后切 HEADING 直航段收尾。现状补充：Cloverleaf 四个转向段同向（`dMax` 符号决定左/右旋四叶，总航向变化 4×270° 净回 ψ0，几何闭合）；HEADING_REACHED 触发带方向敏感（P12：`cumHeading` 须沿阈值符号方向达到幅值才切段，防反舵顺漂预触发）。
- 方案 B（未采用）：将 270° 转向拆为 3 个 90° HEADING 子段，每段最短路径方向与期望一致。

7.4.6 与 PD 控制器的指令衔接



注意：RUDDER 段旁路了 PD 控制律，但**不得旁路安全限幅**——恒舵角指令仍须经舵角/舵速限幅（设计 §4.10）。已落地（P7 整改，§10.5-P7 已关闭）：`autopilot_core` 中 RUDDER 段经 `pd_.saturate(cmd.rudder)` 施加幅值/舵速限幅。

7.5 测试要点

用例	场景	预期
空序列	setLegs({})	done()==true，update 输出安全值
单段 TIME	threshold=1.0, dt=0.02	恰好 50 拍后切段
cumHeading 跨零	航向 359→1→3	累计 +4°，无 358° 跳变
Williamson 全程	Nomoto 仿真闭环	三段依次推进，末段航向 → ψ0+180°±2°
Zigzag 计数	N=5	恰好 10 段后 done

用例	场景	预期
Circles 一圈	恒舵角仿真	累计 360° 触发完成
Cloverleaf 卡死复现	当前实现 + 仿真	复现 §7.4.5 缺陷（回归用例，整改后转为通过）
低频调用防御	1 Hz 调用 + 20°/s 转向率	文档化折叠风险（接口约束用例）

---

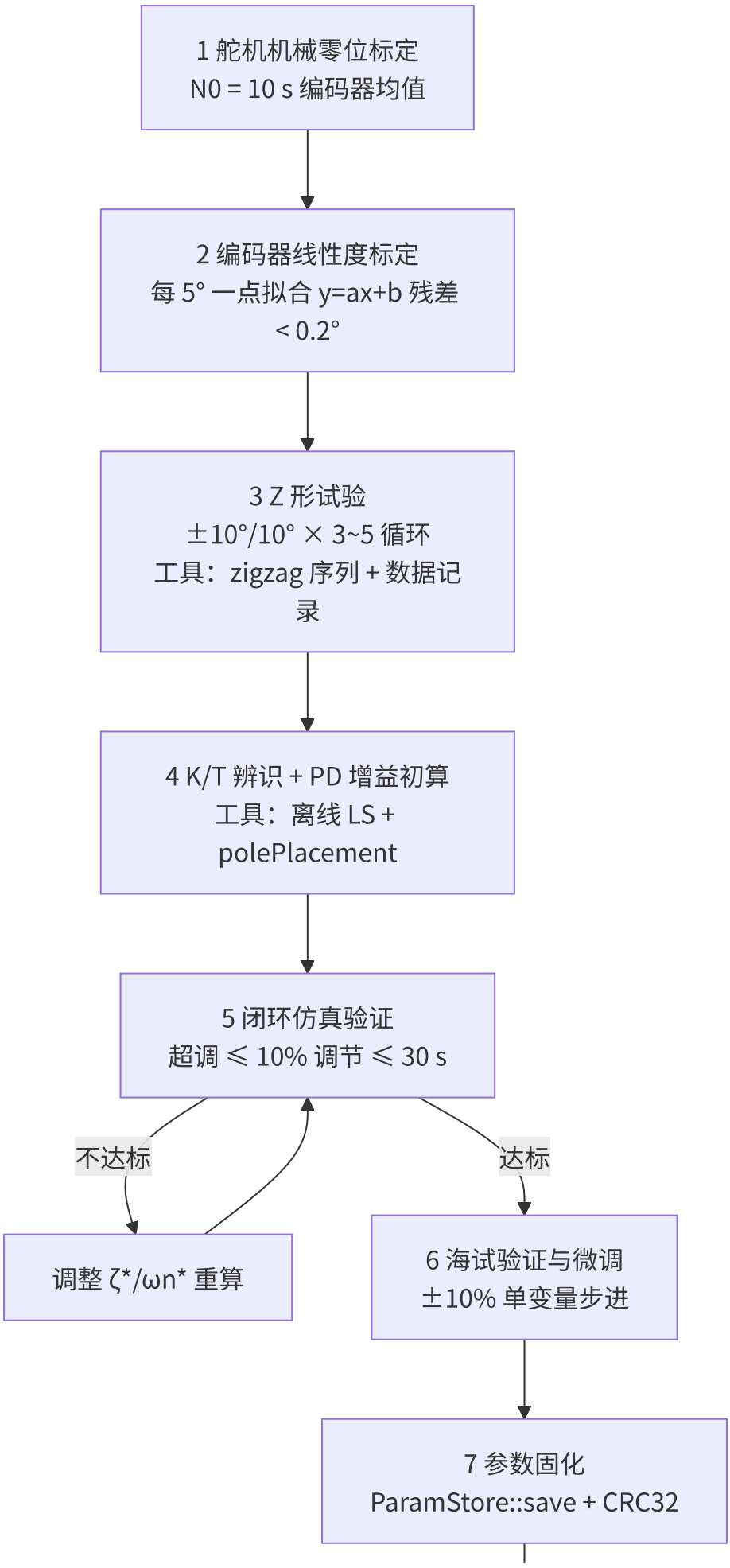
8. 参数存储与标定流程开发

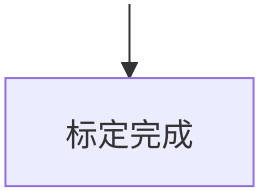
8.1 设计依据

设计文档 §3.9（控制算法标定）、§3.9.8（标定参数存储）、§3.10.9（自动优化参数存储与日志）。实现文件：`control/include/param_store.hpp`。

8.2 标定总流程与算法开发的关系

标定是"用试验数据喂算法"的过程，七步流程中有四步直接消费本工程交付的算法/工具：


1 舵机机械零位标定
N0 = 10 s 编码器均值
2 编码器线性度标定
每 5° 一点拟合 y=ax+b 残差
< 0.2°
3 Z 形试验
±10°/10° × 3~5 循环
工具：zigzag 序列 + 数据记
录
4 K/T 辨识 + PD 增益初算
工具：离线 LS +
polePlacement
5 闭环仿真验证
超调 ⩽ 10% 调节 ⩽ 30 s
不达标
达标
调整 ζ*/ωn* 重算
6 海试验证与微调
±10% 单变量步进
7 参数固化
ParamStore::save + CRC32



标定步骤	消费的模块/工具	交付里程碑
步骤 3	`ManeuverSequencer::zigzag` + 日志记录	M4
步骤 4	离线辨识工具（§5.3 路线 A）+ `AutoTuner::polePlacement`	M2/M3
步骤 5	闭环仿真环境（§10.2）	M1
步骤 7	`ParamStore`	M2

8.3 参数集与 EEPROM 区块映射

```
struct ControlParams {
    // 区块 A：执行器标定
    double rudderZero, rudderLeft, rudderRight;  // N0 / NL / NR
    double ddotMax;                               // 舵速限幅 (deg/s)
    // 区块 B：模型参数
    double K, T;
    // 区块 C：PD 增益
    double Kp, Kd;
    // 区块 D：增益调度表（P8 新增）
    std::vector<GainPoint> schedTable;
    // 元信息
    uint32_t version, timestamp, crc;
};
```

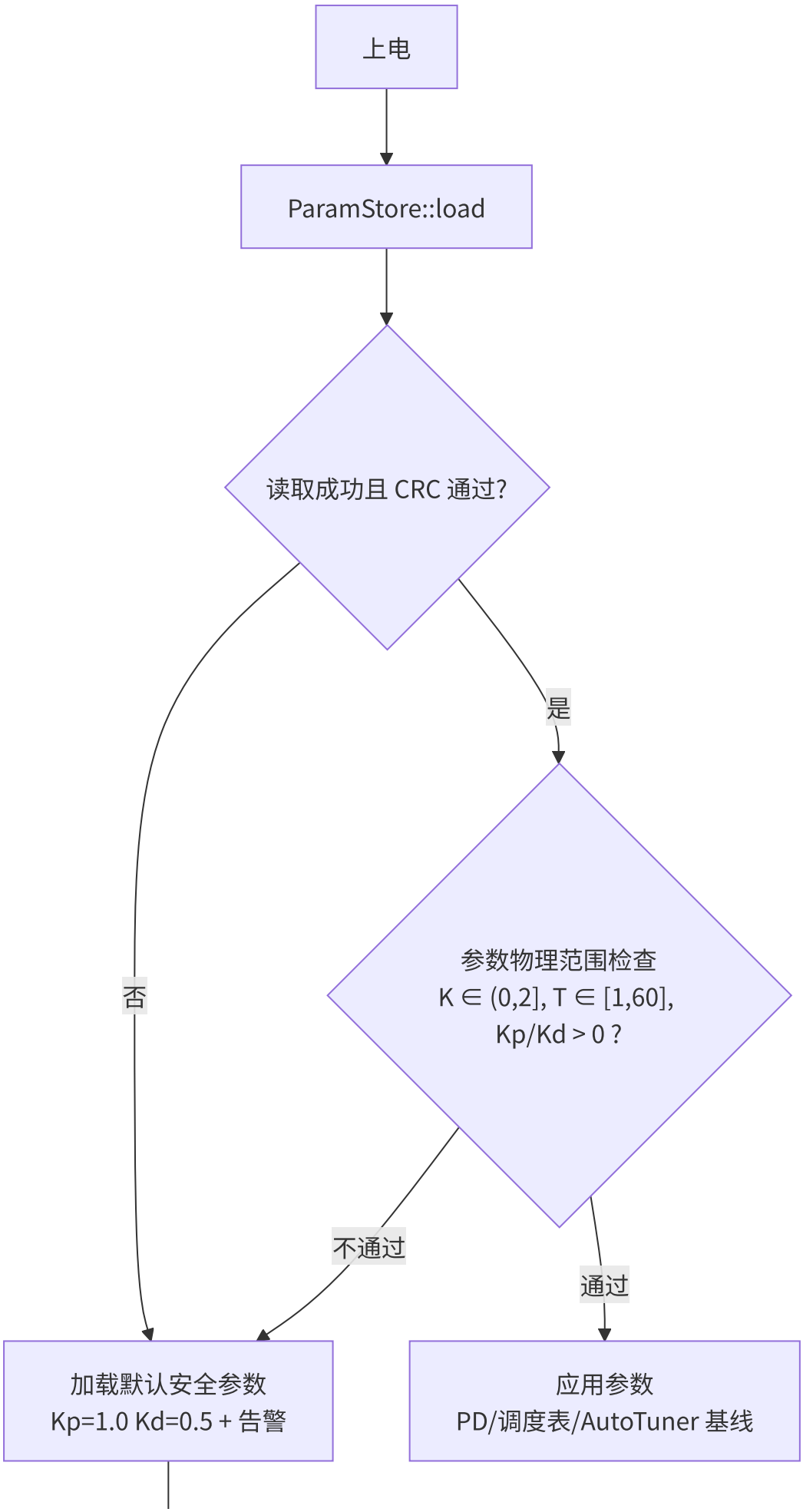
设计区块	内容	当前实现	差距
区块 A	N0/NL/NR、舵速限幅	已含	—
区块 B	K、T	已含	—
区块 C	Kp、Kd	已含	—
区块 D	增益调度表	已含（P8 新增，§10.5-P8 已关闭）	—

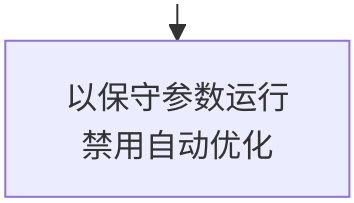
设计区块	内容	当前实现	差距
元信息	版本 + 时间戳 + CRC	已含	时间戳由注入时钟自动填写（P8）

8.4 CRC 与版本机制

- CRC32 采用 IEEE 802.3 多项式 `0xEDB88320`，逐 bit 实现（无查找表，牺牲速度换 ROM 空间，仅在保存/加载时调用，可接受）。
- `save()`：版本号 +1 → 计算 CRC → 序列化写入；`load()`：读取 → CRC 校验，失败返回 false 由上层加载默认安全参数并告警（设计 §3.9.8）。
- **人工标定优先级高于自动优化**（设计 §3.10.8）：人工标定写入后，AutoTuner 的基线（`setBaseline`）必须重置为标定值。

上电加载流程：


上电
ParamStore::load
读取成功且 CRC 通过?
是
否
参数物理范围检查
K ∈ (0,2], T ∈ [1,60],
Kp/Kd > 0 ?
不通过
通过
加载默认安全参数
Kp=1.0 Kd=0.5 + 告警
应用参数
PD/调度表/AutoTuner 基线



（范围检查为增补要求，P8 已实装 `validateRange()` ——K∈(0,2]、T∈[1,60]、Kp>0、Kd≥0、舵速限幅 1~30°/s 等，由上层在 `load()` 成功后调用； `load()` 本身仍只做 CRC 校验，§10.5-P8 已关闭。）

8.5 序列化实现的风险与整改

旧实现 `serialize()` 用 `memcpy(this)` 截取 `sizeof(ControlParams) - 4` 字节，存在三个问题（§10.5-P8，**已关闭——P8 整改已落地**，逐字段显式小端序列化）：

1. **CRC 字段自包含**： `ControlParams` 按 8 字节对齐后 crc 位于截取范围内， `updateCrc()` 计算摘要时包含旧 crc 值、写入新值后 `verifyCrc()` 重算的摘要即与存储值不符——**save 后立即 load 会校验失败**。整改（已落地）：序列化范围显式排除 crc 字段（逐字段序列化，crc 追加在 payload 末尾）。
2. **依赖结构体内存布局**：填充字节内容未定义，同一份参数两次序列化可能产生不同 CRC；跨编译器/ABI 不可移植。整改（已落地）：逐字段小端序列化。
3. **load 读取长度**与 save 写入长度不一致（ `sizeof` vs `sizeof-4+4` 的布局差异），依赖填充巧合。整改后统一为显式格式（现状）： `[8×double 区块A/B/C][version u32][timestamp u32][schedCount u32][区块 D 调度表逐点]` + 末尾 crc u32，长度随调度表条数变化。

8.6 标定操作规程要点（供试验大纲引用）

步骤	前置条件	通过判据	数据留存
零位/满舵	舵机离合脱开，静水系泊	10 s 采样标准差 < 2 LSB	N0/NL/NR 原始码
线性度	每 5° 一点	拟合残差 < 0.2°	全部点位码值
舵速	±20° 阶跃	舵速上限 = 40°/t_r，与铭牌值一致（±20%）	阶跃响应曲线
Z 形试验	海况 ≤ 1 级、场地 Z1 区、船速按 D1 计划	3~5 完整循环，r 信噪比 > 10 dB	δ(t)/r(t)/ψ(t) 50 Hz 原始记录
辨识	Z 形数据	残差 RMS < 5% · r_max	K/T + 残差报告

步骤	前置条件	通过判据	数据留存
仿真	辨识 K/T	§1.5 四项指标全达标	仿真曲线 + 指标表
海试微调	场地 D1~D2	航向误差 1σ ≤ 1°（静水）	每次调整的增益与结果
固化	以上全部通过	save 后回读 CRC 通过	EEPROM 镜像备份

8.7 测试要点

用例	场景	预期
读写回环	save → load（内存模拟 IO）	参数逐字段一致，CRC 通过（P8 整改后该用例通过，作为回归用例保留）
CRC 篡改	写入后翻转任意 1 bit	load 返回 false
版本递增	连续 save 3 次	version 严格 +1 递增
IO 失败	ReadFn/WriteFn 返回 false	load/save 返回 false，不崩溃
默认参数回退	load 失败路径	上层拿到保守默认参数

---

9. 核心集成与主循环开发

9.1 设计依据

设计文档 §5（定位与控制联合工作流）：§5.1 主循环、§5.2 模式切换、§5.3 定位失效降级、§5.4 数据流时序。实现文件：`control/include/autopilot_core.hpp`。

9.2 集成接口

`AutopilotCore::step(SensorInput, dt) → 目标舵角` 是算法层与 BSP 层唯一的运行时边界：

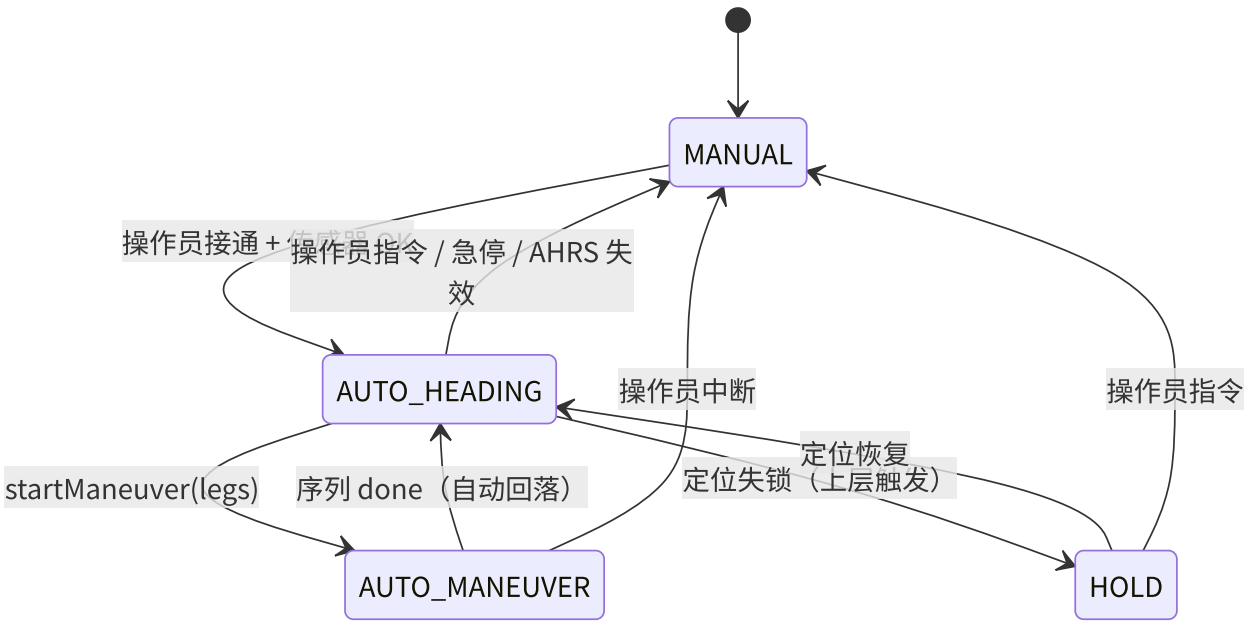
```
struct SensorInput {
    double headingDeg;   // AHRS 艏向（已融合 GNSS 校正后的值）
    double rateDegS;     // AHRS 转向率
    double rudderDeg;    // 舵角反馈
    double speedKn;      // 船速（GNSS/计程仪）
```

```
    int    seaState;     // 海况 0..3（人工设定或估计）
};
```

边界约定：航向融合（"AHRS 高频 + GNSS 校正"，设计 §5.1）与传感器有效性判断在 BSP/融合层完成，`headingDeg` 进入算法层时已是可信值；定位失效降级（设计 §5.3）表现为上层切换模式，算法层不感知 GNSS 状态。

辅助接口（R1 深化）：调参器参数经 `setTunerParams(const AutoTuner::Params&)` 整组注入（替代旧的 mutable 引用接口）。自动优化在 core 内只剩"每拍转发"：全部编排（速度门控/辨识评估/ESC 窗口/稳定门/回退/扰动）收在 `AutoTuner::tick()` 内部（§6.2、§9.4）；辨识试验收官 `finalizeIdentTrial` 缩为一行 `tuner_.establishBaseline(idSnapshot())` 映射 OK/FAILED；`setMode` 切出到 MANUAL/HOLD 时调 `tuner_.suspend()`（清测量状态、保留基线）。

9.3 模式状态机



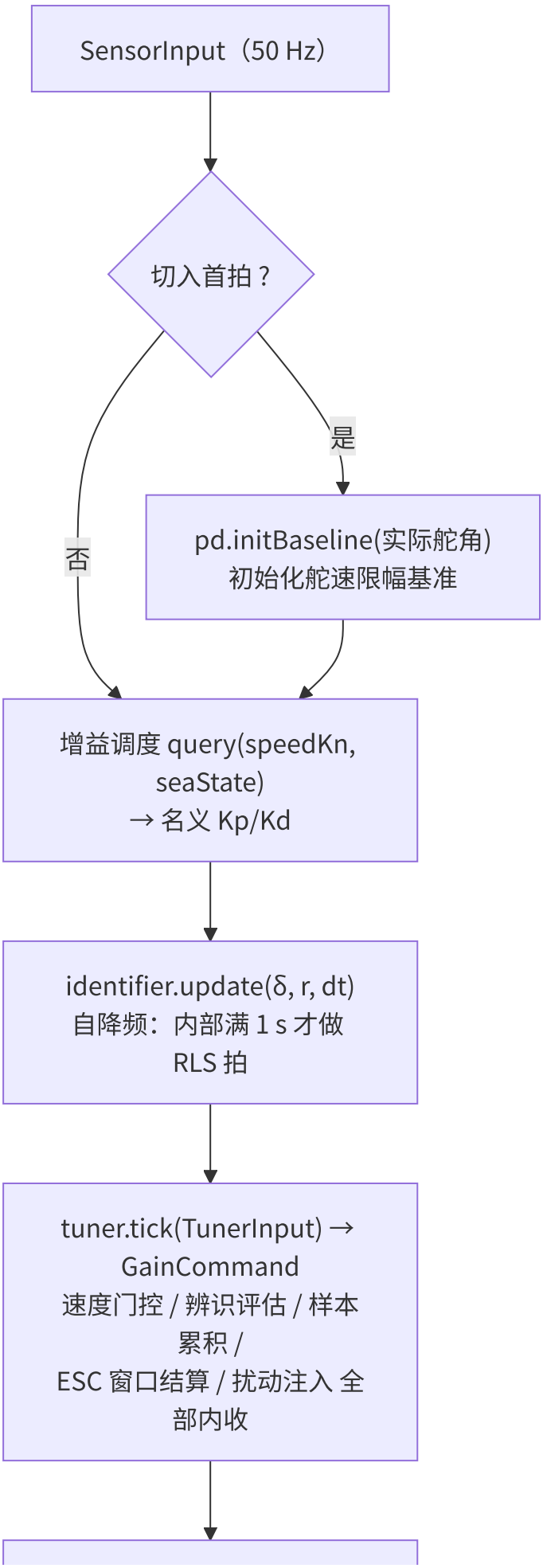
各模式输出语义：

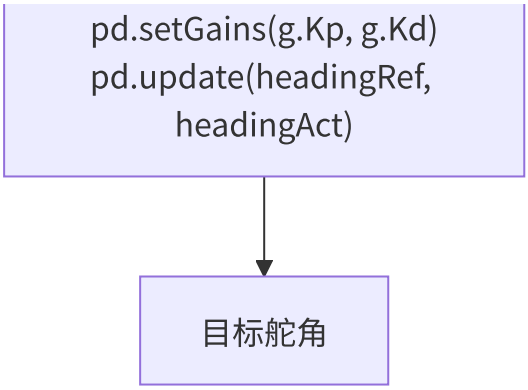
模式	输出	说明
MANUAL	透传 `rudderDeg` 反馈	算法层不干预
AUTO_HEADING	PD(headingRef, headingAct)	含增益调度 + 自动优化
AUTO_MANEUVER	序列指令（HEADING 段走 PD / RUDDER 段直出）	done 后自动回 AUTO_HEADING

模式	输出	说明
HOLD	冻结上一拍舵角指令	P9 整改已落地：与设计"保持"语义一致（§10.5-P9 已关闭）

模式切换的状态清理（P9 整改已落地）：切入新模式前 `setMode()` 清理上一模式残留——`pd_.reset()` 清微分历史与舵速限幅基准、取消进行中的辨识试验，切出到 MANUAL/HOLD 时 `tuner_.suspend()`；切入 AUTO_HEADING 的首拍再以实际舵角初始化舵速限幅基准（P11 `pendingBaseline_` / `initBaseline`）。否则切换瞬间会产生舵角跳变。

9.4 AUTO_HEADING 周期内流程


SensorInput（50 Hz）
切入首拍？
是
否
pd.initBaseline(实际舵角)
初始化舵速限幅基准
增益调度 query(speedKn, seaState)
→ 名义 Kp/Kd
identifier.update(δ, r, dt)
自降频：内部满 1 s 才做 RLS 拍
tuner.tick(TunerInput) → GainCommand
速度门控 / 辨识评估 / 样本累积 /
ESC 窗口结算 / 扰动注入 全部内收



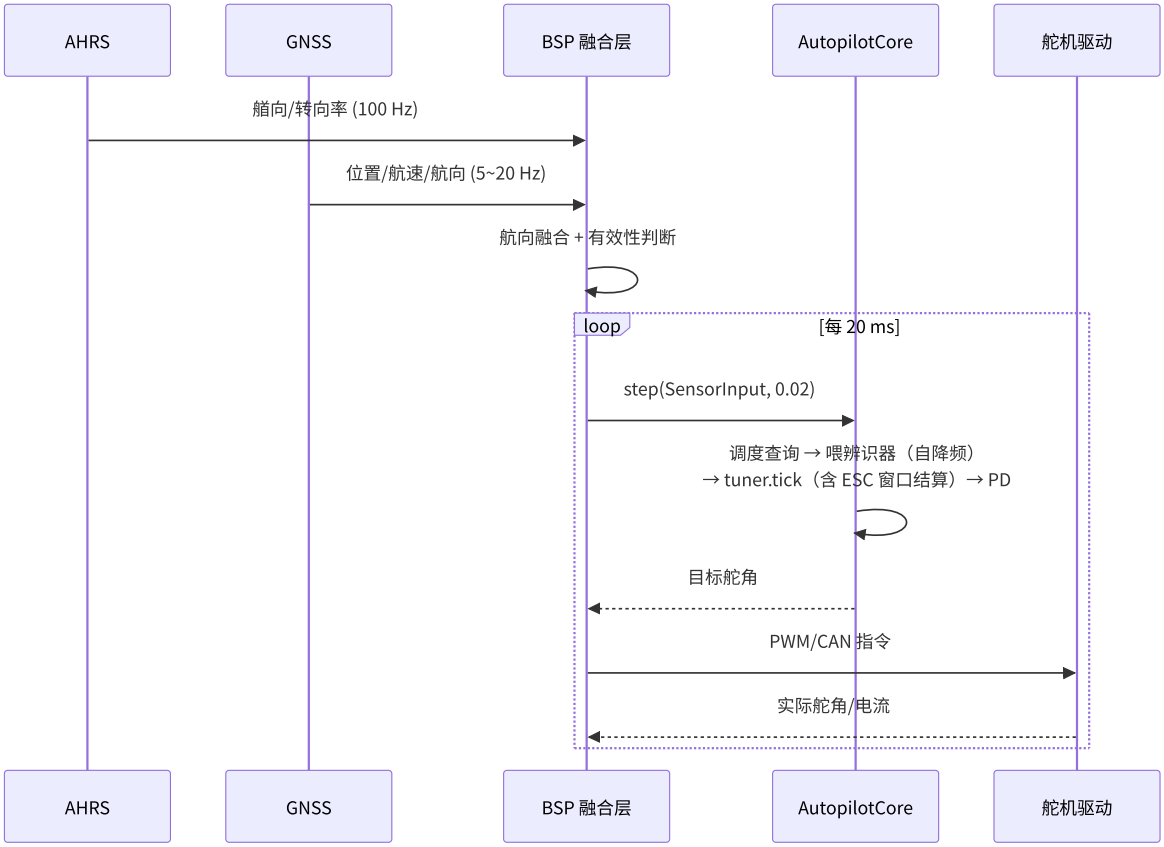
R1 深化：自动优化的编排已全部收进 `tuner_.tick()`（接口现状见 §6.2），上图即 `headingControlStep` 的全部内容（约 15 行）——core 不再维护任何调参计数器与增益副本。ESC 窗口结算同样内收：tick 按 `windowSec/dt` 换算拍数自维护 `winCounter_`，满窗调私有 `settleWindow_()`（§6.4.2 的时间尺度分离不变），原外部 60 s 调度器随之取消。

9.5 各模块调用节拍

模块	节拍	调用者	备注
`AutopilotCore::step`	20 ms（50 Hz）	BSP 主循环	唯一运行时入口
`PdController::update`	20 ms	step 内	—
`GainSchedule::query`	20 ms	step 内	查表开销 O(表长)
`NomotoIdentifier::update`	20 ms 调用	step 内	辨识器内自降频 beatSec=1 s：满一拍才做 RLS，Δt 自测（R1，P3 根治）
`AutoTuner::tick`	20 ms	step 内（AUTO_HEADING）	R1 单入口：速度门控/辨识评估/样本累积（含解调）/扰动注入
ESC 窗口结算（`settleWindow_`，私有）	60 s 窗口	tuner 内 `winCounter_` 满窗自触发	原 core `escTrigger` 内收；时间尺度分离不变
`ManeuverSequencer::update`	20 ms	step 内（AUTO_MANEUVER）	—

模块	节拍	调用者	备注
`ParamStore::save/load`	事件驱动	上电 / 标定完成 / 参数更新	不进主循环

9.6 数据流时序



R1 深化后不再有外部 60 s 调度器：ESC 窗口结算由 `tick()` 内部按窗口拍数自触发（§9.5），主循环时序只剩 20 ms 一拍。

9.7 实时性预算

50 Hz 周期内算法层最坏执行时间预算 < 2 ms（周期的 10%）：

环节	预算	依据
增益调度查表	< 10 μs	14 条线性扫描
RLS 更新（触发拍）	< 50 μs	2×2 矩阵定长运算
极点配置 + 稳定性检查	< 10 μs	纯标量运算

环节	预算	依据
accumulate + 队列维护	< 20 µs	固定容量环形数组（P2 已落地，编译期容量 3000，主循环无堆分配）
PD update	< 10 µs	标量运算
合计	**< 100 µs**	裕量 20 倍

`computeJ()`（3000 样本遍历）只在窗口结算拍执行（每 60 s 一拍；R1 起结算在 `tick()` 内完成，计入该拍主循环开销，预算仍有充足裕量）。

9.8 测试要点

R1 深化后 core 只做"模式状态机 + 机动编排 + 每拍转发"，用例相应全部经公有接口（`step` / `setMode` / `startIdentifyTrial` + 只读查询）驱动，现状 17 用例：

用例	场景	预期
DefaultModeIsManual	构造后	默认 MANUAL
ManualPassesRudderThrough	MANUAL 下 step	透传舵角反馈
AutoHeadingProducesNonzeroRudder	切入 AUTO_HEADING 并给航向误差	输出非零舵角
SetModeResetsState（P9）	模式切换	PD 状态清理，切换瞬间输出连续
HoldFreezesLastRudder（P9）	切入 HOLD	冻结上一拍舵角指令
HoldThenManualClears	HOLD → MANUAL	状态清理，恢复透传
HighSpeedDisablesTuner	speedKn > 15 kn	tick 速度门控停用调参，仅调度表增益生效
LowSpeedEnablesTuner	speedKn ⩽ 15 kn	调参器启用

用例	场景	预期
ManeuverCompletesToAutoHeading（P11）	Williamson 执行完	完成拍输出 PD 闭环结果（非 0 瞬跳），自动回 AUTO_HEADING 并保持当前航向
EnterAutoHeadingSeedsRudderBaseline（P11）	舵面在 10° 时切入 AUTO_HEADING	首拍以实际舵角为舵速限幅基准，不从 0° 爬升
EscDitherReachesPdGainsAfterBaseline	合成 Z 形数据驱动在线辨识建基线	基线建立后 ESC 载波真实叠加到 PD 实际增益（±escAmp 调制）
NoBaselinePassesScheduleGainsThrough（替代 RunEscTuningNoBaselineNoop）	无激励、基线未建立	tick 不结算不扰动，调度增益原样直通到 PD
IdentUpdatesDoNotOverwriteEscGains（P12）	基线建立后辨识估计漂移	漂移 ±25% 阈值内增益钉在旧基线（增益所有权）；越阈才显式重基线跳到新配置
EscFallbackRestoresBaselineOnOscillation（P12）	注入峰-峰 12° 振荡误差	窗口结算被稳定门拒绝，增益 revert 回极点配置基线
OscDetectorStarvedDuringReferenceMotion（P12）	航向参考每 15 s 翻转	机动期间振荡检测不喂入（幅值恒 0）；参考稳定 30 s 后恢复喂入
ManeuverHeadingLegAppliesScheduledGains（P13）	机动 HEADING 段	应用调度表增益，不停留默认增益
IdentTrialIdentifiesAndAppliesGains（P14）	startIdentifyTrial + Nomoto 仿真对象	收官 establishBaseline 成功：identTrial=OK、辨识 valid、增益 = 极点配置值、回 AUTO_HEADING

---

10. 测试与验证计划

10.1 单元测试（V 模型右侧第 1 级）

各模块用例已列于 §3.6/§4.6/§5.6/§6.6/§7.5/§8.7/§9.8，汇总覆盖要求：

模块	用例数（R1 现状）	关键覆盖
angle_utils	7	±180° 边界（左开右闭）、跨零、大角度折叠
pd_controller	12	处理链顺序、首拍、死区、双限幅、量化、setGains/setDt 清状态语义
gain_schedule	7	段边界、越界防御、外推、插值连续性
nomoto_identifier	9	收敛性（含 SelfDecimatesAtAnyCallRate 自降频）、激励门限、坏数据防御、物理合理性、极点配置退化/clamp/往返
auto_tuner	17	极点配置、四条件稳定门、振荡检测、环形数组，及 tick 行为：载波直通/窗口结算钳制/ESC 闭环收敛/回退/重锚清积分（§6.6）
maneuver_sequencer	14	三类触发、切段清零、六机动推进、缺陷回归
param_store	7	回环、CRC 篡改、版本、IO 失败
autopilot_core	17	模式切换、机动回落、tick 每拍转发、增益所有权、速度门控、辨识试验（§9.8）
tuner_debug_api	13	单功能独立调用、与在线路径逐位一致、force* 确属旁路、setParams 复位语义（§12.5）

合计 103 用例（9 个测试文件），Release + ASAN 双工具链全绿。

准出：全部通过 + ASan/UBSan 无报告 + 行覆盖率 ≥ 90%。

10.2 闭环仿真验证（V 模型右侧第 2 级）

仿真被控对象（PC 端，与单测同工程）：

$$\dot{r} = \frac{K\delta_{\text{act}} - r}{T}, \qquad \dot{\delta}_{\text{act}} = \text{clip}\left(\frac{\delta_{\text{cmd}} - \delta_{\text{act}}}{\tau_s},\ \pm\dot{\delta}_{\max}\right), \qquad \dot{\psi} = r$$

- 舵机一阶惯性 τ_s = 0.3 s + 舵速物理限幅；
- 量测噪声：ψ 加 σ = 0.2° 白噪声，r 加 σ = 0.1°/s；
- 干扰：恒值风致力矩（等效舵角 2°）+ 0.1 Hz 波浪扰动。

验证矩阵（每项跑典型船参数 K=0.2/T=8 与边界参数 K=0.5/T=3、K=0.1/T=20）：

验证项	判据（§1.5 / 设计 §3.9.6）
30° 航向阶跃	超调 ⩽ 10%，5% 调节时间 ⩽ 30 s
航向保持（带干扰）	稳态误差 ⩽ 1°，操舵频次 ⩽ 6 次/分钟
增益调度切换	船速斜坡 2 → 24 kn，无输出跳变 > 1°
RLS + 极点配置闭环	从偏差 50% 的初始 K/T 出发，10 min 内恢复设计指标
ESC 长期运行	2 h 仿真 J 无发散，增益始终在界内
六机动形态	轨迹积分绘图与设计 §4 图样一致
稳定性回退注入	人为注入坏辨识值，监控拒绝并回退，无失稳

10.3 HIL 硬件在环（V 模型右侧第 3 级）

- 算法编译进目标 ECU，仿真模型运行在 PC，经 CAN/串口以 50 Hz 闭环；
- 验证项：实时性（step 最坏执行时间 < 2 ms）、定点/浮点一致性（ECU 与 PC 结果偏差 < 0.01°）、参数固化断电恢复、模式切换与急停响应 < 100 ms（设计 §1.3）；
- 设计 §6.3 要求：RLS 收敛性与 ESC 稳定性先在 HIL 验证，再分速段海试。

10.4 海试验证（V 模型右侧第 4 级）

对应设计文档 §7.11 场地分区与日程：

日程	区域/限速	验证内容	本文档关联
D1	Z1，⩽ 8 kn	Z 形试验、K/T 辨识、PD 整定	§5、§8 标定流程
D2	Z2，⩽ 15 kn	Williamson/U-Turn/Zigzag/Circles	§7 机动序列
D3	Z3，⩽ 25 kn	高速航向保持、高速机动、自动优化分速段验证	§6、§9
D4	Z4，⩽ 25 kn	紧停、应急退出、降级切换、Search 开环	§9 模式状态机

海试数据全程 50 Hz 记录（δ_cmd/δ_act/ψ/r/U/增益/模式/触发原因），格式与 §3.10.9 优化日志一致，支持岸基回放与回滚分析。

10.5 已知实现差距与待完善项清单

代码走查（2026-07-30，对照 `control/include/` 当前版本）发现的差距，按优先级排序。**P1~P3 必须在 M5 集成联调前关闭**：

编号	优先级	位置	问题	整改方案
P1	高	`autopilot_core.hpp` step()	每周期 `pd_.setParams()`，而 `setParams` 内部 `reset()` 清空微分历史与舵速限幅基准，导致微分项恒为首拍 0、舵速限幅失效	PdController 增加 `setGains(Kp,Kd)` 轻量接口（不清状态）；dt 单独设置 ——**已关闭（P1 整改已落地）**：`setGains` / `setDt` 每拍轻量更新不清状态，主循环不再调 `setParams`（§4.5）
P2	高	`auto_tuner.hpp`	ESC 为简化解调（dJ×sin 相关），且滑动窗口用 `std::deque`（主循环堆分配）	HIL 收敛慢则升级标准 ESC 结构；deque 改定长环形数组 ——**已关闭（P2+P11 整改已落地）**：窗口为定长环形数组（编译期容量 3000，无堆分配）；ESC 已升级标准结构（每拍扰动注入 + 因果解调 + HPF/LPF，§6.4.2）
P3	高	`autopilot_core.hpp`	`identifier_.update()` 按 50 Hz 调用且 Params.dt=0.02 与设计 1 s 节拍不符，差分信噪比低	step 内加 1 s 降频计数器，Identifier dt 配 1.0 ——**R1 深化已关闭**：辨识器内自降频（`beatSec`=1 s），Δt 自测，`Params.dt` 字段删除，调用方无需计数器
P4	高	`autopilot_core.hpp` / `auto_tuner.hpp`	`stabilityCheck` 的 oscAmplitude 实参恒为 0，四条件之振荡检测永真；振荡检测算法未实装	开发 30 s 窗口航向误差峰峰值统计，接入 C3 条件与 §6.4.3 触发矩阵 ——**已关闭（P4 整改已落地）**：`OscillationDetector` 过零结算峰-峰幅值实测并接入 C3；P12 起参考稳定满 30 s 才喂入（§6.5）
P5	中	`gain_schedule.hpp`	注释声称插值实为阶跃查表；段切换有增益跳变	速度维线性插值或增益斜坡限速（§4.4）——**已关闭（P5 整改已落地）**：速度维段中点锚定线性插值，段切换跳变消除（§4.2/§4.4，回归用例 InterpolationContinuityAtBoundary）

编号	优先级	位置	问题	整改方案
P6	中	`nomoto_identifier.hpp` / `auto_tuner.hpp`	辨识结果无物理范围检查；polePlacement 对 K≈0 返回 Kp=Kd=0（开环危险）	K∈(0,2]、T∈[1,60] 门限；polePlacement 改返回 bool，失败保持原增益；Kd clamp ⩾ 0 ——**已关闭（P6 整改已落地）**：`valid()` 门限、polePlacement 返回 bool（退化 false）、Kd clamp ⩾ 0 均已实装（§5.5、§6.3）
P7	中	`maneuver_sequencer.hpp` / `autopilot_core.hpp`	Cloverleaf/Search 转向段 HEADING+270°/90° 阈值与 PD 最短路径矛盾（序列卡死）；RUDDER 段输出未过限幅；U-Turn 方向不受控	§7.4.5 方案 A 或 B；RUDDER 段包一层幅值/速率限幅；U-Turn 加方向引导 ——**已关闭（P7 整改已落地）**：三机动转向段改 RUDDER 恒舵角（方案 A）；RUDDER 段经 `pd_.saturate()` 限幅；U-Turn 方向由舵角符号强制；P12 追加方向敏感触发（§7.4.4~§7.4.6）
P8	中	`param_store.hpp`	serialize 含 crc 字段自身与填充字节（save 后 load 校验失败、跨平台不可移植）；缺区块 D（调度表）；load 无物理范围检查；timestamp 未填	逐字段显式序列化（§8.5）；扩展区块 D；加载后范围检查（§8.4） ——**已关闭（P8 整改已落地）**：逐字段小端序列化（CRC 排除自身）、区块 D 调度表、注入时钟自动填 timestamp、`validateRange()` 范围检查均已实装（§8.3~§8.5）
P9	低	`autopilot_core.hpp`	`setMode` 无状态清理（切换瞬间舵角跳变）；HOLD 输出 0 与设计"保持"语义待评审；`baselineKp_` 初值 0 时 revert 会回退到 0	setMode 内 reset PD 并以实际舵角初始化限幅基准；HOLD 语义评审；baseline 未初始化时 revert 改为保持当前值 ——**已关闭（P9 整改已落地）**：`setMode` 状态清理 + HOLD 冻结上一拍舵角；R1 起 revert 仅在基线建立后可达，回退到 0 的路径不存在（§9.3）
P10	低	`maneuver_sequencer.hpp`	`zigzag()` / `cloverleaf()` 内 `cum` 变量未使用；`zigzag(psi0,...)` 的 psi0 未参与序列构造	清理死代码；zigzag 增加末段"回 ψ0 保持"可选项 ——**已关闭**：`cum` 死变量已不存在；psi0 与"回 ψ0 保持"可选项随 P12 整改落地（`zigzag(..., returnToStart)`，保持段目标=psi0，§7.4.1）

10.6 验收放行结论模板

里程碑	放行条件	证据
M1~M4	§10.1 单测准出 + 对应模块仿真项通过	单测报告 + 仿真曲线
M5	P1~P3 关闭 + §10.2 全矩阵通过 + §10.3 HIL 通过	HIL 报告
海试放行	M5 通过 + P4~P8 关闭 + 场地验收清单（设计 §7.12）全勾选	海试大纲评审记录

---

11. 可视化调参工具开发

11.1 定位与现状

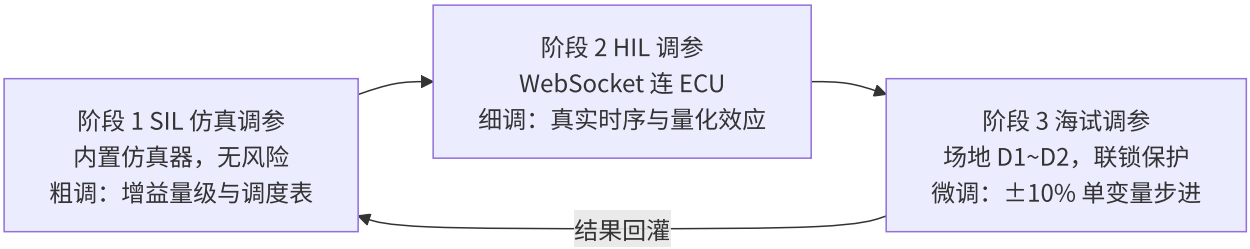
定位：可视化调参工具（`control/ui/debug_ui.html`，配套说明 `control/ui/README.md`）是贯穿 M1~M5 的开发支撑项目——参数整定、算法验证、HIL 联调、海试调参都以它为人机界面。工具内的算法实现是 C++ 头文件的 JS 镜像，**仅用于调试可视化，不可用于实船控制**（实船闭环必须走 C++ ECU）。

现有能力（V1.0，933 行单文件 Web 应用，浏览器直接打开）：

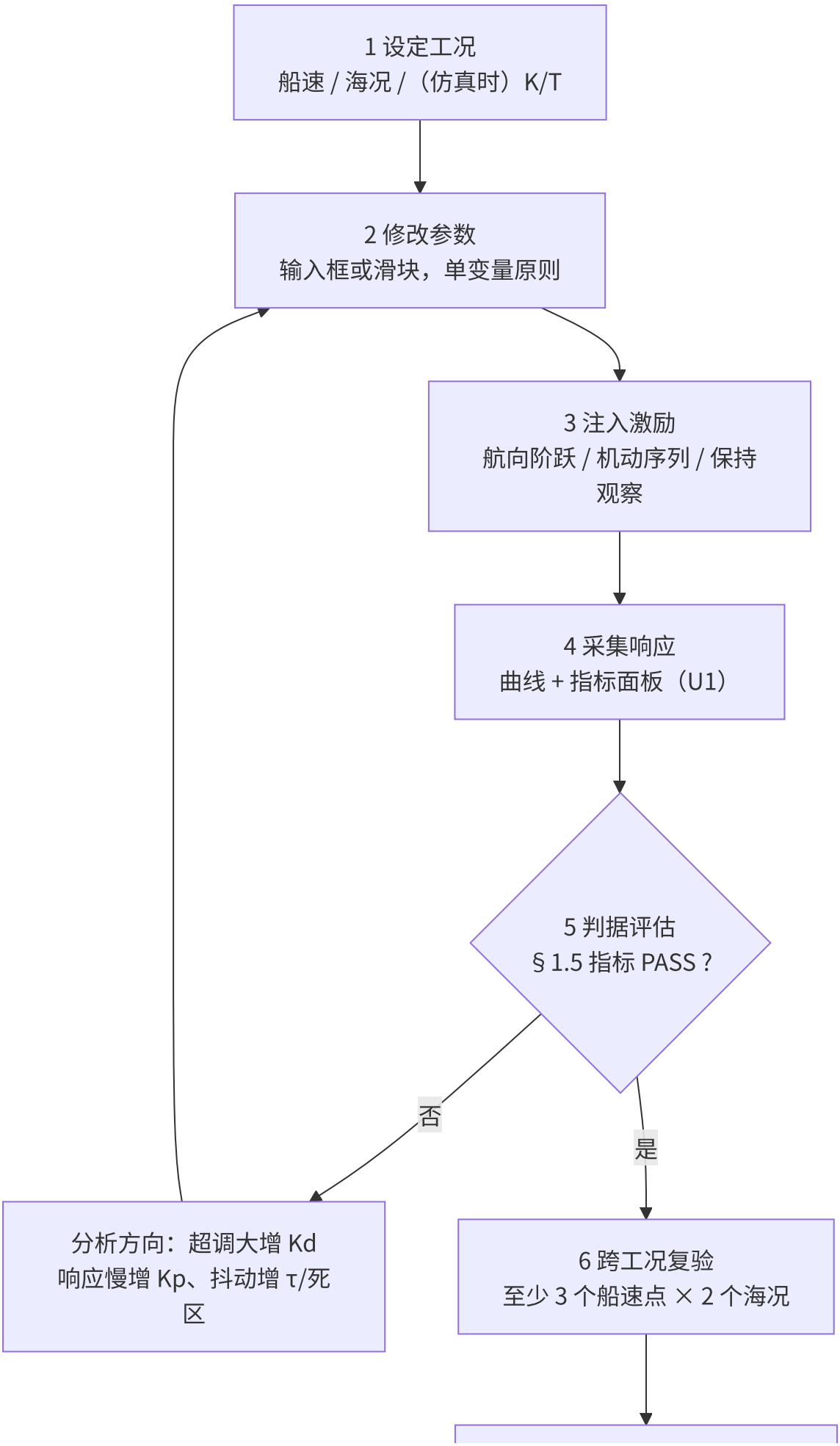
能力	说明	覆盖章节
参数在线编辑	PD / 调度表 / 辨识器 / 优化器 4 组参数，与 C++ `Params` 逐字段映射	§3~§6
实时曲线	航向、舵角、误差/转向率三路 Canvas，50 Hz 刷新	—
实时滑块	Kp / Kd / 航向目标 / 船速 / 海况 五路拖动即生效	§3、§4
机动触发	6 种预置机动一键启动 + 段推进显示	§7
自动优化观察	$\hat{K}$ / $\hat{T}$ / J / ζ / ω_n 实时显示，启用 / 禁用 / ESC 单步 / 回退	§5、§6
双模式运行	内置 Nomoto 仿真器（SIL）/ WebSocket 对接真实 ECU	§10.2、§10.3
数据导出	CSV / JSON（含参数快照），供 MATLAB/Python 离线分析	§10.4

11.2 可视化调参总流程

调参按 **SIL 仿真 → HIL 半实物 → 海试** 三阶段递进，参数只能"向上带"（上一阶段的结果作为下一阶段初值），不允许跳级：



每一轮调参遵循统一的闭环（任一阶段通用）：


1 设定工况
船速 / 海况 /（仿真时）K/T
2 修改参数
输入框或滑块，单变量原则
3 注入激励
航向阶跃 / 机动序列 / 保持
观察
4 采集响应
曲线 + 指标面板（U1）
5 判据评估
§ 1.5 指标 PASS ?
否
是
分析方向：超调大增 Kd
响应慢增 Kp、抖动增 τ/死
区
6 跨工况复验
至少 3 个船速点 × 2 个海况

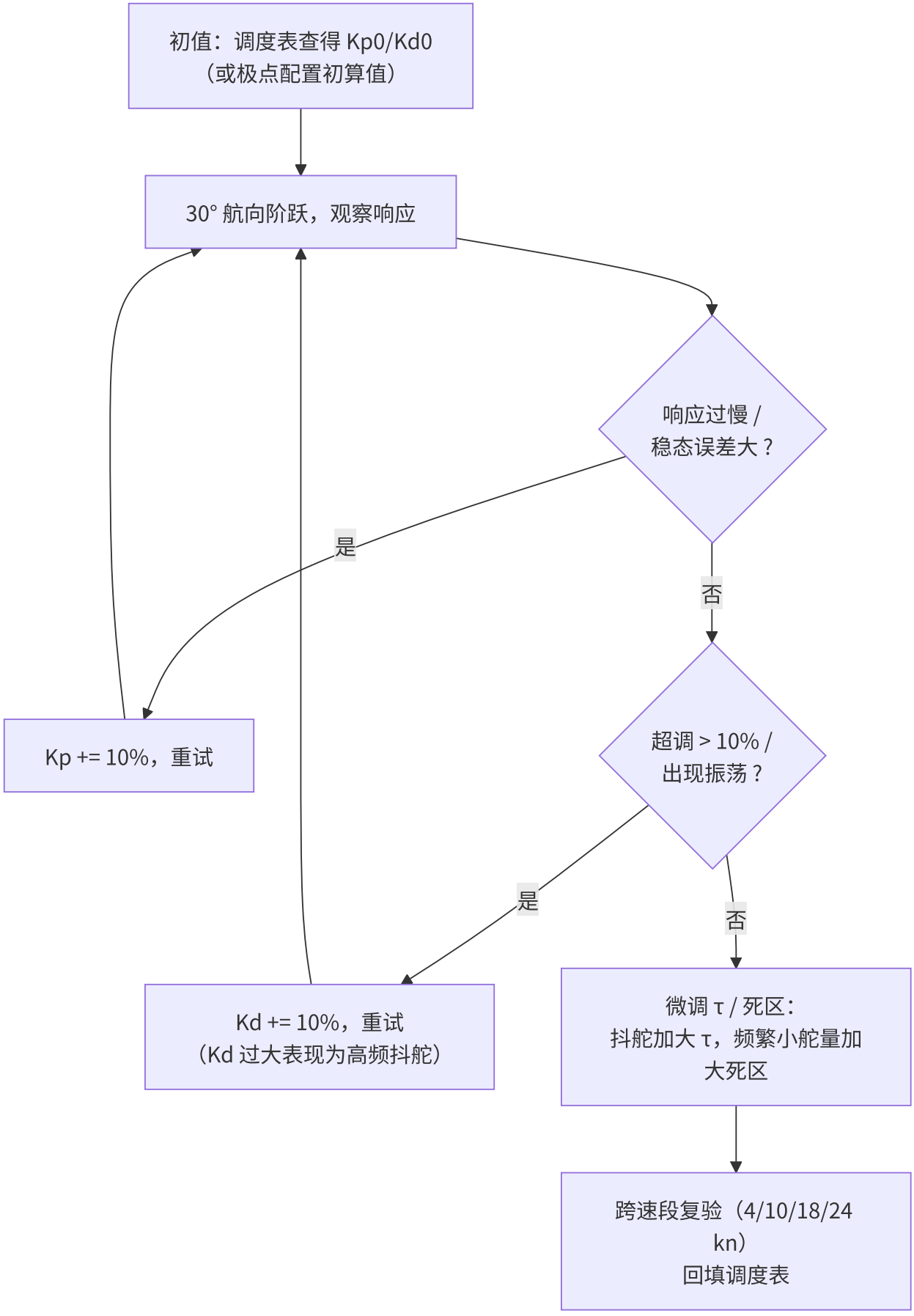
7 固化与记录
下发 ECU ParamStore + 会话日志（U6）

单变量原则：每轮只改一个参数（与设计文档 §3.9.7 一致，海试阶段步长 ±10%）；改多个参数后无法归因，会话日志（U6）按次记录以便回滚。

11.3 分对象调参细则

11.3.1 PD 手动整定（M1 阶段主任务）

从调度表初值出发的七步整定法：



各参数的调节现象速查表（调参时对照曲线判断方向）：

参数	调大的现象	调小的现象	建议步长
Kp	响应快、超调增大、临界时等幅振荡	响应慢、稳态误差大	±10%
Kd	超调受抑、噪声下高频抖舵	阻尼不足、二次超调	±10%
τ	抖舵消失、微分作用滞后	微分灵敏、噪声放大	±0.05 s
deadzone	操舵频次降、小误差不纠	频繁小幅操舵	±0.1°
ddMax	转向凌厉、舵机负荷大	响应钝、大误差收敛慢	按舵机标定值

11.3.2 增益调度表整定

1. 在 4 个速度段代表点（2 / 7 / 14 / 22 kn）分别按 §11.3.1 整定出局部最优 Kp/Kd；
2. 在 UI"调度"标签回填表格并应用；
3. 用船速滑块做 2 → 24 kn 斜坡扫描，确认段切换处无输出跳变超标（§10.2 判据 ≤ 1°）；
4. 海况 2~3 行：在仿真器叠加波浪扰动后重复上述流程，通常取静水增益的 60~75%。

11.3.3 辨识与优化参数调参要点

参数	现象与调节方向	观察窗口
λ（遗忘因子）	偏小：K̂/T̂ 跟踪快但噪声大；偏大：平滑但滞后。从 0.98 起 ±0.005	K̂/T̂ 曲线（U5 平面）
P0	过小：初期收敛慢；过大：初期跳变剧烈。1e4~1e6 均可接受	前 30 个有效样本
δ/r 激励门限	过低：直航噪声污染辨识；过高：长期无更新	"激励"状态灯占空比
escStep η	过大：J 振荡不收敛；过小：收敛慢。0.01~0.05	J 历史趋势
escAmp	过大：扰动本身劣化航向精度；过小：梯度信噪比不足	航向误差 1σ
ζ*/ωn*	直接决定响应风格：ζ* 大更稳，ωn* 大更快	U5 约束平面落点

11.4 工具增强开发项（V1.1，U1~U7）

现状差距：V1.0 只能"看曲线、凭经验判断"，指标计算、试验流程、对比归因、安全联锁均靠人工。V1.1 按以下 7 项增强，编号纳入 MV 里程碑（§2.2/§2.3）：

编号	项目	动因	交付随
U1	性能指标面板	§1.5 判据自动评估，替代人工读图	M1
U2	一键阶跃试验	标准化激励，试验可重复	M1
U3	参数扫描与 A/B 对比	网格寻优 + 归因对比	M3
U4	数据回放	海试数据事后复盘	M4
U5	ζ-ωn 约束平面	稳定性监控可视化	M3
U6	调参会话日志与回滚	单变量原则的记录载体	M4
U7	在线调参安全联锁	海试在线改参的安全前提	M5

U1 性能指标面板

对最近一次阶跃/保持试验自动计算并按 §1.5 阈值显示 PASS/FAIL：

指标	计算方法	阈值
超调量	(max(ψ) − ψ_ref) / 阶跃幅值 × 100%	⩽ 10%
5% 调节时间	误差最后一次进入 ±5% · 阶跃幅值 的时刻	⩽ 30 s
稳态误差 1σ	进入稳态后误差标准差	⩽ 1°
操舵频次	rudderCmd 变化方向翻转次数 / 分钟（幅值 > 量化步长才计）	⩽ 6 次/min
ITAE J	复用 `computeJ` 同款公式	趋势参考

U2 一键阶跃试验

按固定脚本执行，消除人工操作差异：稳定保持 10 s（误差 1σ < 0.5° 才继续）→ 施加 +30° 阶跃 → 采集 60 s → 施加 −30° 回程 → 采集 60 s → 自动弹出 U1 报告并可导出。左右双向各一次，检查不对称性（不对称提示舵零位偏差，联动 §8 标定）。

U3 参数扫描与 A/B 对比

- **网格扫描**（仅 SIL 模式）：设定 Kp ∈ [a,b]×N 点、Kd ∈ [c,d]×M 点，逐点静默仿真固定场景（30° 阶跃 + 60 s 保持），输出 J 的 N×M 热力图，点击热力图单元格即载入该组参数。为 §11.3.1 手动整定提供初值定位。

- **A/B 对比**：保存当前响应为基线曲线（灰色叠加显示），改参后新响应与基线同屏对比，指标面板双列显示差值。

U4 数据回放

导入 §10.4 海试记录或 UI 自身导出的 CSV/JSON，以可调倍速重放曲线并重算 U1 指标。用途：海试当晚复盘、缺陷回归（如 §7.4.5 序列卡死的现场数据回放）、向评审提供证据曲线。

U5 ζ-ωn 约束平面

以 ζ 为横轴、ωn 为纵轴绘制稳定性监控（§6.5）的可视化：

- 画出约束边界 ζ = 0.6（左禁区）与 ωn = 0.5 rad/s（上禁区）；
- 实时描点 ($\hat{\zeta}$, $\hat{\omega}$n)（由当前 Kp/Kd 与 $\hat{K}/\hat{T}$ 按 §6.3 验证公式反算），带 60 s 轨迹尾迹；
- 目标点 (ζ*, ωn*) 用十字标注。点落入禁区即红色告警，与 `stabilityCheck` 拒绝行为一致——调参者可在增益被拒绝**之前**直观看到趋势。

U6 调参会话日志与回滚

每次"应用参数"自动追加一条会话记录：

```
{ "ts": 1722310000, "scope": "pd", "old": {"Kp":1.5,"Kd":0.6}, "new": {"Kp":1.65,"Kd":0.6},
  "metrics": {"overshoot":8.2,"ts5":22.1,"ess":0.6,"freq":4.1}, "stage": "HIL" }
```

- 侧栏时间线展示，任意条目一键恢复该组参数（含下发 ECU）；
- 导出为 JSON 附入试验报告，与设计文档 §3.10.9 优化日志格式对齐（字段命名一致，便于岸基工具统一解析）。

U7 在线调参安全联锁（ECU 已连接时强制生效）

联锁项	规则	依据
幅度限制	单次下发增益变化 > 20% 时拒绝并提示分步	设计 §3.10.7 单次 10% 的 2 倍上限（人工权限放宽）
高速禁改	ECU 上报船速 > 15 kn 时禁用增益下发（灰化按钮）	设计 §3.10.6 高速去激活
确认机制	下发前弹窗显示 old → new 差异，二次确认	—
应答超时回滚	2 s 内未收到 `params_ack` 或 `accepted:false`，UI 状态回滚并告警	README §4.4 协议

联锁项	规则	依据
机动中禁改	模式为 AUTO_MANEUVER 时禁用全部参数下发	设计 §4.10
紧急恢复	常驻"恢复安全参数"按钮：一键下发 EEPROM 默认安全组	设计 §3.9.8

11.5 与各里程碑的调参任务对照

阶段	使用工具能力	调参对象	产出
M1 PD 仿真验证	U1 + U2 + 滑块	Kp/Kd/τ/死区（§11.3.1）	各速段初版增益
M2 辨识调试	优化标签 + U5	λ/P0/激励门限（§11.3.3）	辨识器参数
M3 自动优化	U3 + U5	ζ*/ωn*/ESC 参数	优化器参数 + J 热力图
M4 机动验证	机动触发 + U4	机动幅值/阈值（dMax、δ_z 等）	机动参数表
M5 HIL	全部 + U7	全参数细调	HIL 调参报告
海试 D1~D2	U1/U2/U6/U7	±10% 单变量微调（设计 §3.9.7）	固化参数 + 会话日志

11.6 验收判据

编号	验收判据
U1	对已知解析响应的仿真数据，指标计算误差 < 2%；PASS/FAIL 与人工判读一致
U2	连续 3 次一键试验，同参数下指标离散度 < 3%（可重复性）
U3	10×10 网格扫描 < 60 s 完成；热力图最优点与手动整定结果偏差 < 15%
U4	导入 1 h 海试 CSV（18 万样本）加载 < 5 s，回放流畅无丢点
U5	描点值与 C++ `stabilityCheck` 反算值一致（1e-6）；禁区告警与拒绝行为同步
U6	100 条会话记录回滚任意一条，参数与记录逐字段一致
U7	六项联锁逐项注入违规操作，全部被拦截且有明确提示；断链时自动回滚验证通过

---

12. 实船调试接口开发

12.1 设计依据与定位

实船调试/标定需要绕开 `tick()` 的固定编排（§6.2、§9.4），对单一环节做激励—观测：只验极点配置数值、只打一拍载波看执行器响应、只喂误差看振荡检测读数、只把当前窗口的 J 读出来。为此在 M3 之后补充交付**实船调试接口**，交付物一览：

交付物	文件	角色
调试接口门面	`control/include/tuner_debug_api.hpp`	`TunerDebugApi` 把 `AutoTuner` 的公有动作与私有机制全部摊平为独立方法，每个方法对应一个且仅一个功能，互不影响、任意顺序调用
命令行工具	`control/tools/tuner_debug_cli.cpp`	逐行命令封装 `ar_tuner_debug`，可经 ssh/串口终端在船上直接使用
实时监控脚本	`control/tools/tuner_debug_monitor.py`	驱动 CLI 轮询 `status`，matplotlib 实时曲线 + 命令转发 + CSV 落盘
端到端验收脚本	`control/tools/tuner_debug_accept.py`	驱动 CLI 对 9 个功能组 49 项断言，退出码可直接接入 CI
单元测试	`control/tests/test_tuner_debug_api.cpp`	13 个用例，纳入 §10.1 测试体系（见 §12.5）

实现机制：`AutoTuner` 仅增加一行 `friend class TunerDebugApi` 声明，调试方法直接调用同一批私有函数（同一参数、同一行为），**不改动任何在线路径**。使用约束（与实船安全直接相关，已写入头文件注释）：

1. 本接口只用于调试/标定，**不得接入正常航行控制回路**——`force*` 系列旁路稳定门，正常路径的安全保证全部失效；
2. 除 `force*` 系列外，其余方法与其在 `tick()` 内部的行为逐字一致，观测结果可代表在线行为（有单测保证，§12.5）；
3. 所有方法均无堆分配、无 IO，可在 ECU 上经串行/以太网命令通道驱动。

12.2 构建：AR_BUILD_TOOLS 与编译产物

`control/CMakeLists.txt` 新增开关，与 `AR_BUILD_TESTS` / `AR_BUILD_SIM` 并列、默认开启：

```
option(AR_BUILD_TOOLS "Build debug/commissioning tools" ON)
```

`AR_BUILD_TOOLS=ON` 时 `add_subdirectory(tools)`，编译产物为单一可执行文件 **`build/tools/ar_tuner_debug`**（仅链接 header-only 的 `ar_headers` INTERFACE 库，无第三方依赖）：

```
cd control/build && cmake .. && make ar_tuner_debug   # 默认即构建
cmake -DAR_BUILD_TOOLS=OFF ..                         # 裁剪发布镜像时可整体关掉
```

12.3 ar_tuner_debug 基本用法

交互模型：每行一条命令，每条命令恰好调用 `TunerDebugApi` 的一个功能，可任意顺序单独调用。输出协议：成功 `OK ...`、失败/拒绝 `ERR <原因>`；观测类命令输出 `key=value` 序列。启动时按 `isatty` 判别——交互终端打印横幅与 `>` 提示符；脚本/管道模式输出保持纯净（无横幅无提示符），上位机逐行解析：

```
./build/tools/ar_tuner_debug              # 交互（可经 ssh/串口终端）
./build/tools/ar_tuner_debug < cmds.txt   # 脚本模式
```

典型会话（行尾注释为输出要点）：

```
> pole 0.2 8                    # OK Kp=1.600000 Kd=8.600000（默认 ζ=0.85 ωn=0.2）
> check 0.2 8 1.6 8.6 0         # OK pass=1（稳定门四条件判定）
> baseline 0.2 8                # OK established=1（一次性建基线）
> forceenabled 1                # 单环节调试前置：绕开速度门控
> dither 1.6 8.6                # OK Kp=... Kd=...（打一拍载波并推进相位）
> accum 1.0 2.0 1               # OK（成本累积 + 振荡检测 + 因果解调）
> step 1.6 8.6                  # OK Kp=... Kd=...（窗末梯度结算）
> settle 1 0.2 8                # OK curKp=... curKd=...（窗末裁决：结算+稳定门+接受/回退）
> status                        # OK curKp=... osc=...（全量快照）
> quit
```

命令分组（`help` 命令可查全表）：

组	命令	对应 TunerDebugApi 方法
观测	`status` / `j` / `osc`	`snapshot()` / `currentJ()` / `oscAmplitude()`
观测（参数）	`params` / `setparam <名称> <值>`	`params()` / `setParams()`
层 2	`pole` / `check` / `baseline` / `eval`	`polePlacement` / `stabilityCheck` / `establishBaseline` / `evaluateEstimate`

组	命令	对应 TunerDebugApi 方法
ESC	`dither` / `accum` / `step` / `settle` / `clamp`	`ditherOnce` / `accumulateOnce` / `escStepOnce` / `settleWindowOnce` / `clampUpdate`
振荡	`oscfeed` / `oscreset`	`oscFeed` / `oscReset`
生命周期	`revert` / `rebase` / `suspend` / `reset`	`revertToBaseline` / `rebaseEsc` / `suspend` / `reset`
主流程	`tick`	`tick()`（与在线路径同一函数）
强制注入	`forcegains` / `forcebase` / `forcephase` / `forcegrad` / `forceenabled`	`forceGains` / `forceBaseline` / `forceEscPhase` / `forceGradient` / `forceEnabled`

实现要点与易错点：

1. **可选参数缺省**：`pole` 的 ζ/ωn 缺省 0.85/0.20；`settle [valid K T]` 三个参数全可选，缺省 `valid=1` + 典型船估计 K=0.2/T=8（`K_SHIP_DEFAULT` / `T_SHIP_DEFAULT`）；`tick` 的辨识快照 4 参数可选（缺省 `excited=0 valid=0`）。
2. **`setparam` 的复位语义**：改写单个参数走 `setParams()`，内部复位调参器——**基线一并清空**（`reset` 不恢复 Params，验收脚本组 6 据此显式恢复默认值，避免参数跨组泄漏）。可选参数名共 17 个：`zeta wn escAmp escFreq escStep hpfTau lpfTau wU wDu maxUpdateFrac absMinUpdate zetaMin wnMax oscMax highSpeedKn windowSec dt`。
3. **单环节调试前置 `forceenabled 1`**：`dither` / `accum` / `step` 依赖 `enabled_`，单独调试时先用 `forceenabled` 置位绕开速度门控，否则命令空转（验收脚本组 3 断言了"未开 enabled 时 dither 不动"）。
4. **`forcebase` 的副作用**：`forceBaseline` 不记录 `baseK_/baseT_`（保持 0），此后 `driftGate=true` 的漂移判据会把任何 K/T 视为超阈值漂移（分母走 floor），下一次评估恒重基线——保守的安全方向，但调用方应知情。
5. **防御式解析**：参数不足、未知参数名、未知命令一律 `ERR` 并附用法提示，不崩溃、不改变调参器状态（验收组 9 覆盖）。

12.4 监控与验收脚本（tools/*.py）

两脚本与既有仿真可视化工具同层，依赖收敛在 `control/tools/requirements.txt`（`matplotlib>=3.5`、`numpy>=1.20`）：

```
pip install -r tools/requirements.txt    # 或 apt 安装 python3-matplotlib python3-numpy
```

`tuner_debug_monitor.py`——调试参数实时监控：驱动 `ar_tuner_debug` 进程（管道请求—响应，线程安全），按固定周期下发 `status` 并解析 `key=value`，用 matplotlib 滚动绘制 4 个子图：增益（当前实线/基线虚线）、ESC 梯度估计（LPF 后）、窗口代价 lastJ、实测振荡 + 载波相位。监控期间在终端输入的任何命令原样转发给 CLI（回显打印），即"一面看曲线、一面做单环节调试"。

```
python3 tools/tuner_debug_monitor.py            # 交互监控（默认找
../build/tools/ar_tuner_debug）
python3 tools/tuner_debug_monitor.py --demo     # 内置激励演示，无实船即可自检监控链路
PLOT_SIM_AGG=1 python3 tools/tuner_debug_monitor.py --demo --duration 15 --out out/monitor.png
# 无 GUI 冒烟/归档
```

选项	含义	默认
`--exe PATH`	`ar_tuner_debug` 可执行文件路径	`../build/tools/ar_tuner_debug`（相对脚本位置）
`--interval S`	`status` 轮询周期 (s)	0.1
`--window S`	曲线滚动时间窗 (s)	60
`--demo`	内置激励序列（`baseline 0.2 8` → `forceenabled 1` → ±2° 方波误差 `accum` + `dither`，每 6 s 一次 `settle`）	关
`--log CSV`	`status` 快照追加写入 CSV（t + 14 个监控字段）	关
`--duration S` / `--out PNG`	固定时长后保存图片退出（无 GUI 冒烟用）	关

实现注意：交互刷新用 `FuncAnimation`，**必须持有引用**（`self._anim`），否则动画对象被 GC、窗口空白（matplotlib 官方陷阱）；监控字段名以 CLI `status` 实际输出的 `key=value` 为准（如基线为 `baseKp` / `baseKd`）。监控器与被调试的调参器是同一进程内实例（经管道请求—响应），船上使用时 `--exe` 指向船上命令通道的封装即可。

`tuner_debug_accept.py`——端到端验收：驱动 CLI 对 9 个功能组逐条断言预期输出，共 **49 项**（每条命令的预期数值均经手工核算）。与 `test_tuner_debug_api.cpp` 互补——单测测 API 层，验收脚本测 CLI 端到端（含参数解析与输出协议）：

```
python3 tools/tuner_debug_accept.py [--exe PATH]   # 全绿输出 "49 PASS / 0 FAIL"
```

退出码全 PASS 为 0、任一 FAIL 为 1，可直接接入 CI。9 个组：①极点配置/稳定门 ②基线管理 ③ESC 各环节 ④振荡检测 ⑤回退/重锚/生命周期 ⑥参数读写 ⑦强制注入 ⑧tick 主流程编排 ⑨输出协议健壮性。

12.5 单元测试纳入测试体系

`control/tests/test_tuner_debug_api.cpp`（13 个用例）按 §10.1 的既有约定纳入：加入 `control/tests/CMakeLists.txt` 的 `AR_TEST_SOURCES` 列表即可，`foreach` 循环自动为其生成独立可执行目标（链接 `ar_headers` + `GTest::gtest_main`）并经 `gtest_discover_tests` 注册（含 ASan/LSan 环境变量），随 `AR_BUILD_TESTS`（默认 ON）与其余测试文件一起构建运行，Release + ASan 双工具链同测。

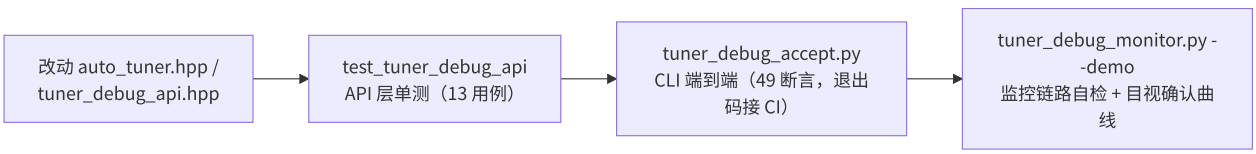
13 个用例中有两条关键保证，是全接口可信度的基石：

用例	保证
`TickThroughApiMatchesDirectTick`	经 API 的 `tick()` 与直接调 `AutoTuner::tick()` 连续 50 拍输出**逐位一致**（DOUBLE_EQ）——调试接口观测即在线行为
`ForceGainsBypassesGate_DebugOnly`	必不过稳定门的增益（Kp<0）经 `forceGains` 照样写入，且基线未被污染——`force*` 确属旁路，且旁路范围受控

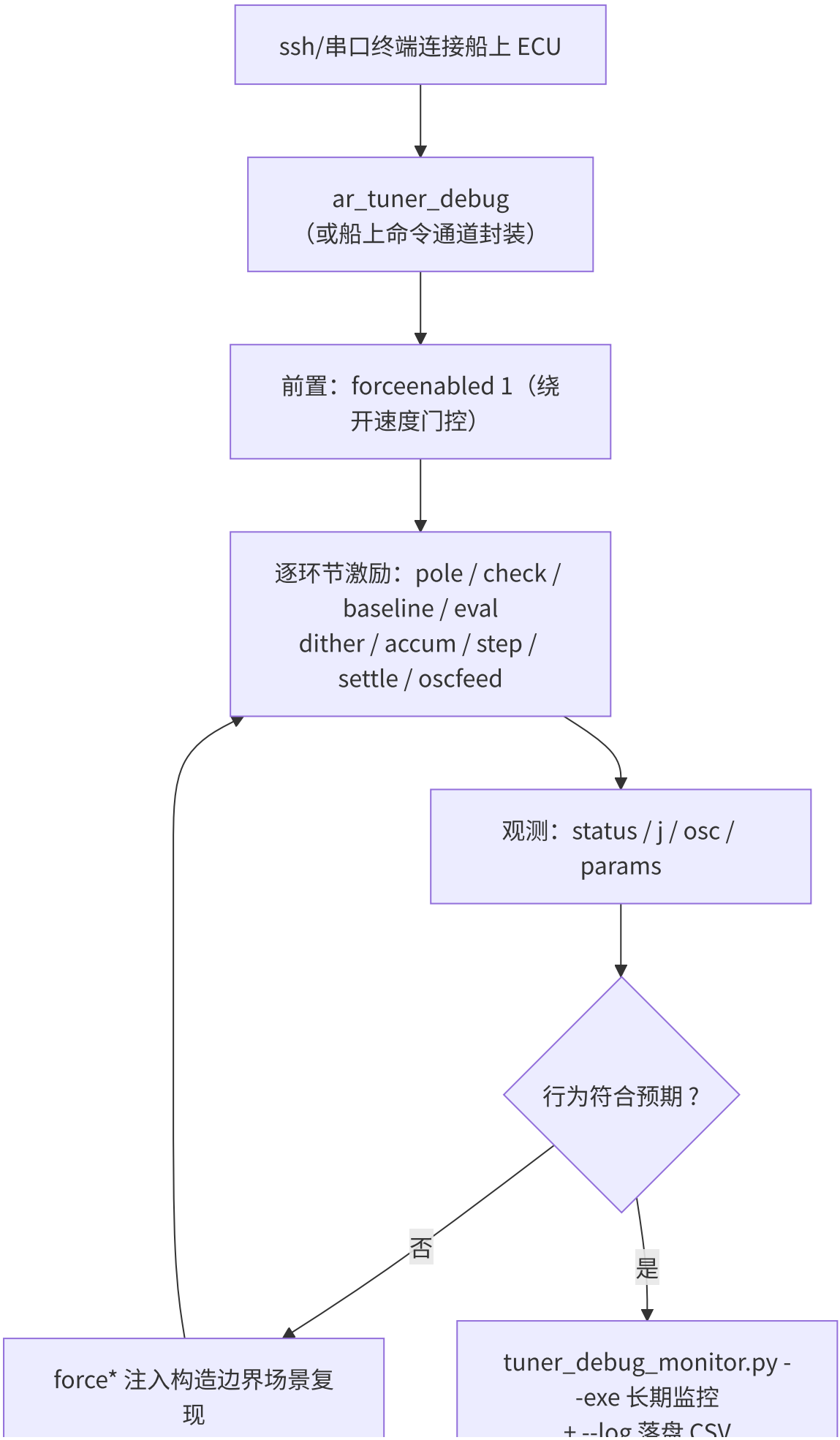
其余用例逐组覆盖：透传一致性（pole/check）、基线建立与快照一致、`evaluateEstimate` 漂移门语义（±25% 阈值内外）、dither 门控与相位推进、accumulate/escStep 独立可调用、clamp 比例上限与绝对下限（P11 锁死回归）、振荡喂入/清零、revert/rebase/suspend 语义、`settleWindowOnce` 接受与回退双路径、`setParams` 复位语义。

12.6 典型开发/调试工作流

开发期（PC，无实船）——改动 AutoTuner 内部机制或调试接口后的回归顺序：



实船调试——以"单环节激励—观测"为主线，以全量快照为遥测：


ssh/串口终端连接船上 ECU
ar_tuner_debug
（或船上命令通道封装）
前置：forceenabled 1（绕
开速度门控）
逐环节激励：pole / check /
baseline / eval
dither / accum / step /
settle / oscfeed
观测：status / j / osc /
params
行为符合预期？
否
是
force* 注入构造边界场景复
现
tuner_debug_monitor.py -
-exe 长期监控
+ --log 落盘 CSV

操作要点：

1. **先验收后调试**：上船先跑 `tuner_debug_accept.py`，确认船上构建的 CLI 行为与 PC 逐条一致；
2. **改参即重来**：任何 `setparam` 都会复位调参器状态（基线清空），改完须重新 `baseline` 建基线再观察 ESC；
3. **落盘复盘**：`--log` 落盘的 CSV 与 `docs/实船调试接口调用文档.md` 的行协议字段一致，事后可按 §11-U4 的回放思路复查；
4. **红线**：`force*` 仅限受控试验（如执行器响应试验用 `forcegains` 注入指定增益）；正常航行控制回路严禁接入本接口（§12.1 约束 1）。

附录 A：术语与符号

符号	含义	单位
ψ / ψ_ref	实际 / 期望航向	deg
e	航向误差（归一化到 (−180°, 180°]）	deg
r	转向率 dψ/dt	deg/s
δ	舵角（右舵为正）	deg
K, T	Nomoto 旋回性 / 追随性指数	1/s, s
Kp, Kd	PD 比例 / 微分增益	—
ζ, ωn	闭环阻尼比 / 自然频率	—, rad/s
λ	RLS 遗忘因子	—
J	ITAE + 控制能耗性能指标	—
ESC	Extremum Seeking Control 极值搜索	—
RLS	Recursive Least Squares 递推最小二乘	—
HIL	Hardware-in-the-Loop 硬件在环	—

— 文档结束 —